Redes Neurais e Deep Learning





Prof. Rodrigo Caropreso
Jul/2019
rcaropreso at gmail

Esta Foto de Autor Desconhecido está licenciado em CC BY-SA

Conteúdo Programático – Aulas 1 e 2

Introdução e Redes Multi-Camadas

- Conceitos Gerais em Aprendizado Profundo
- Perceptron Multi-Camadas;
- Backpropagation e Validação Cruzada.
- Aspectos Gerais de Treinamento Supervisionado

Redes Neurais Recorrentes

- Redes Neurais com Retardos Temporais (TDNN);
- Redes Convolucionais: Conceitos e Exercícios
- Keras – Conceitos e Exercícios

Conteúdo Programático – Aula 3

Redes Neurais Convolucionais

- Redes Neurais Recorrentes
  - Redes de Jordan
  - Redes de Elman
  - Redes LSTM: Conceitos e Exercícios
- Auto Encoders: Conceitos e Exercícios
- GAN's: Conceitos e Exercícios



Redes Neurais Recorrentes

Fonte: http://cdn.jornalgrandebahia.com.br/2018/01/Intelig%C3%AAncia-Artificial-IA-106-500x333.jpg

Redes Neurais Recorrentes: Conceitos

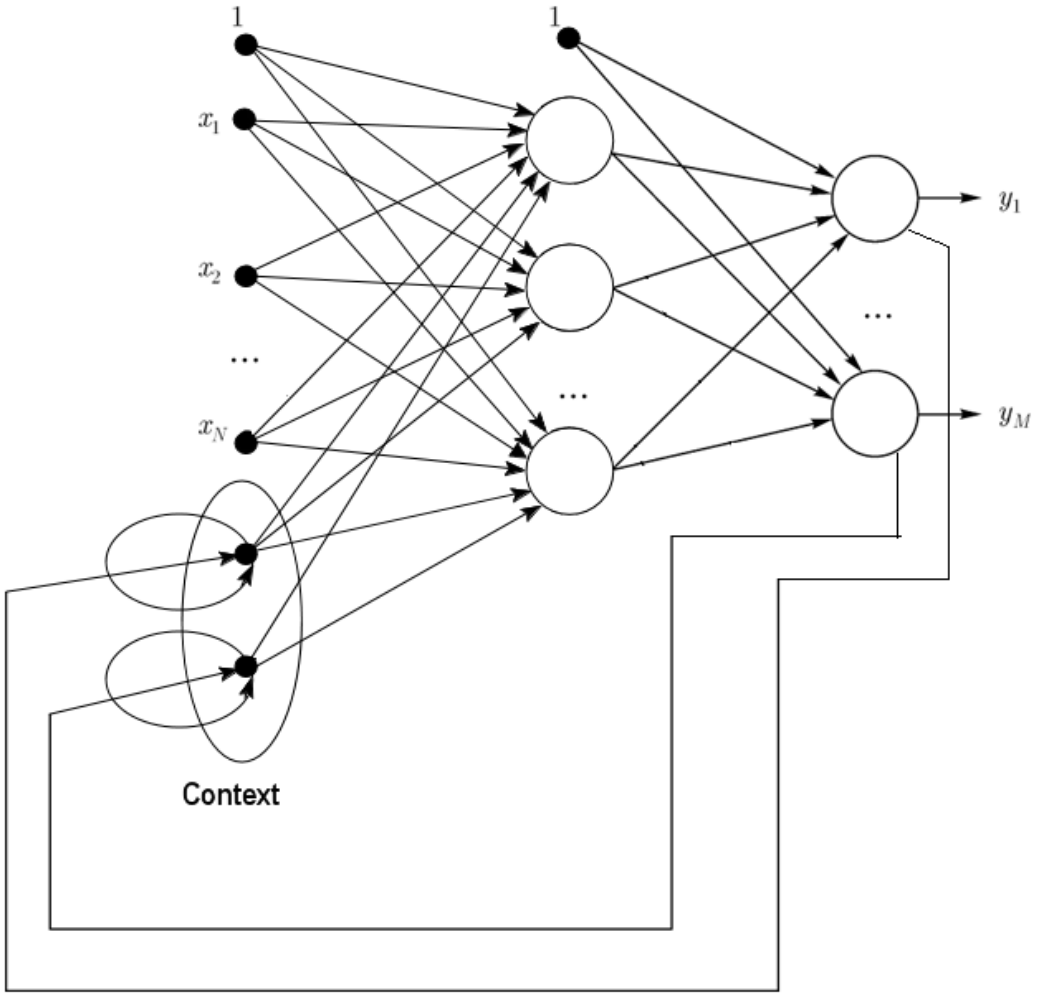


- Diferentemente das arquiteturas tradicionais, a arquitetura com conexões recorrentes possibilita a recuperação de respostas passadas a partir da realimentação de sinais produzidos em instantes anteriores.

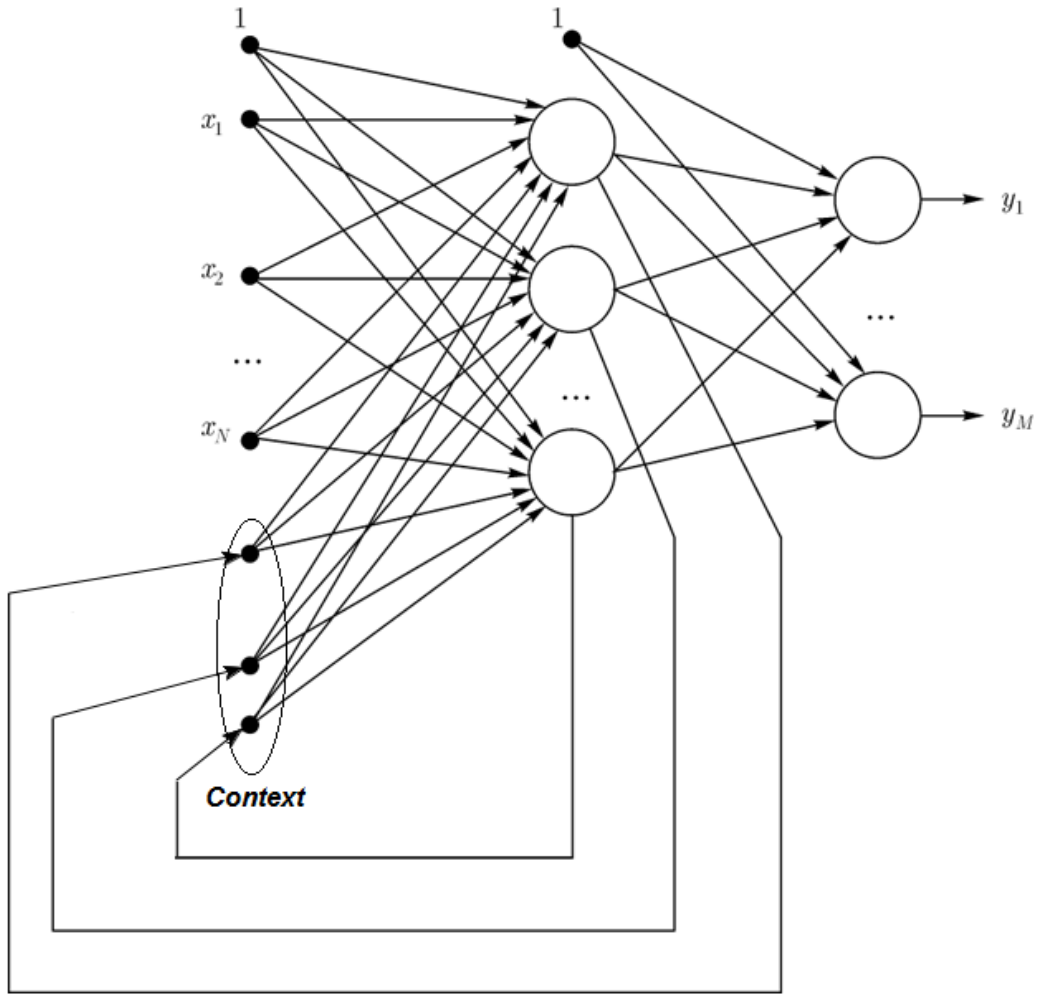
- Pode-se dizer que a configuração recorrente possui memória, sendo capaz de “relembrar” saídas passadas a fim de produzir a resposta atual ou futura.

Redes Neurais Recorrentes: Jordan e Elman

Rede de Jordan



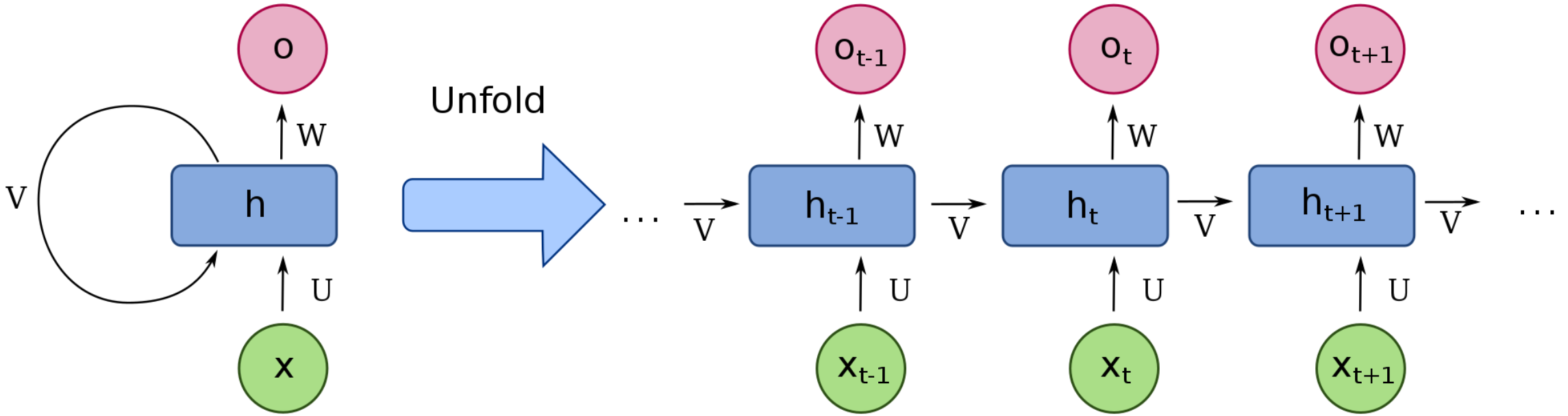
Rede de Elman



https://www.mladdict.com/elman-recurrent-neural-network-simulator

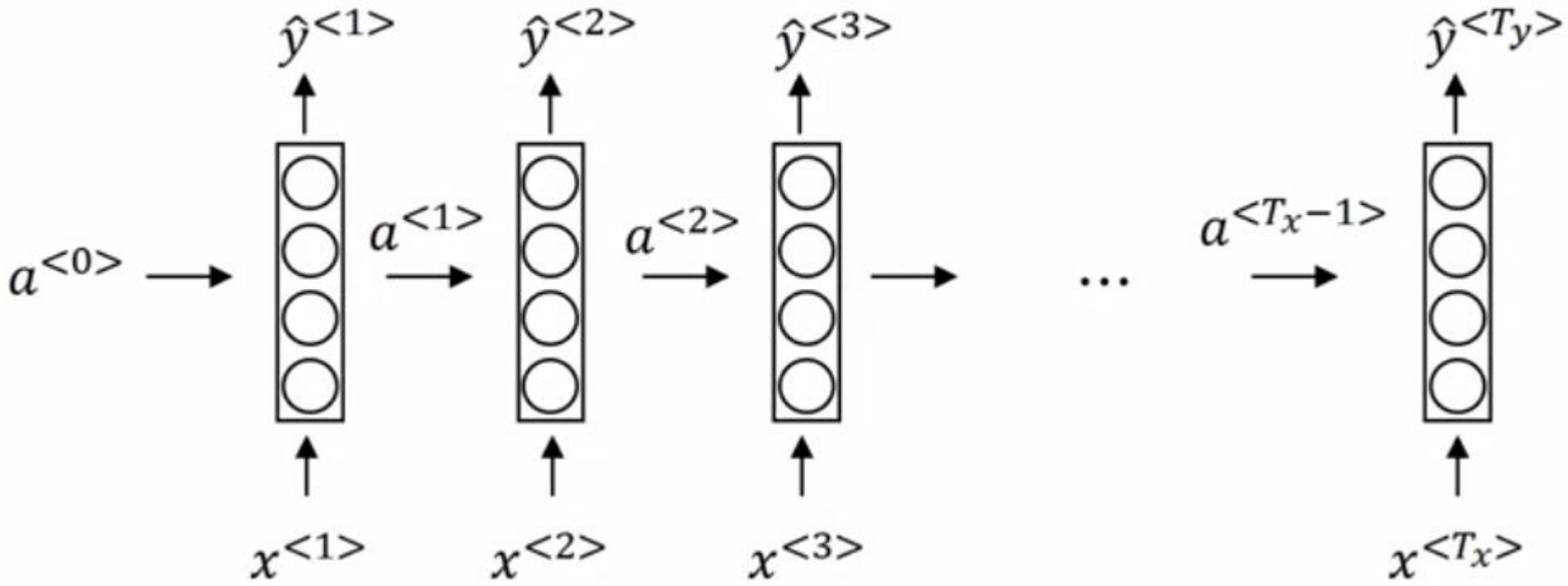
Fonte: https://www.mql5.com/en/articles/1103

Redes Neurais Recorrentes: Aspectos de Treinamento



Esta Foto de Autor Desconhecido está licenciado em CC BY-SA

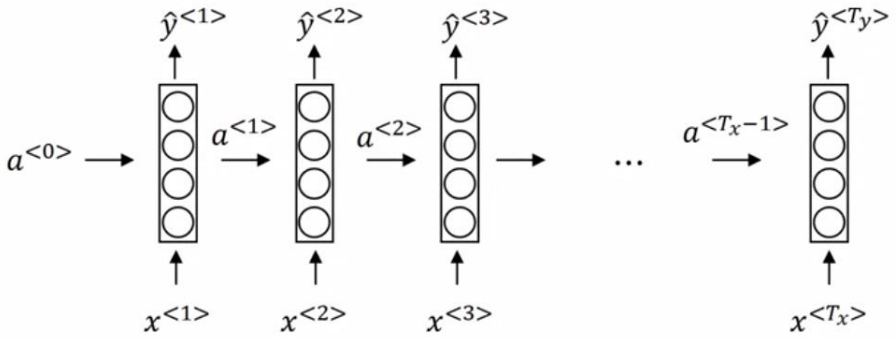
Redes Neurais Recorrentes: Aspectos de Treinamento



- Uma característica que precisa ser destacada aqui é que esta topologia apenas utiliza as informações de instantes anteriores da sequencia.
- Para utilizar as informações de distintos elementos da sequencia devemos recorrer a redes do tipo bi-direcional (BDRNN's).

Fonte: https://www.coursera.org/learn/nlp-sequence-models/lecture/ftkzt/recurrent-neural-network-model

Redes Neurais Recorrentes: Notação

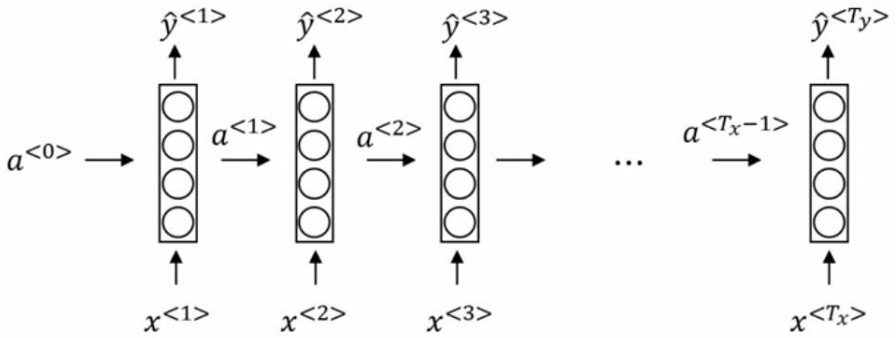


Entradas: $x^{<1>}, x^{<2>}, x^{<3>}, \ldots, x^{<t>}, \ldots, x^{<9>}$

Saídas: $y^{<1>}, y^{<2>}, y^{<3>}, \ldots, y^{<t>}, \ldots, y^{<9>}$

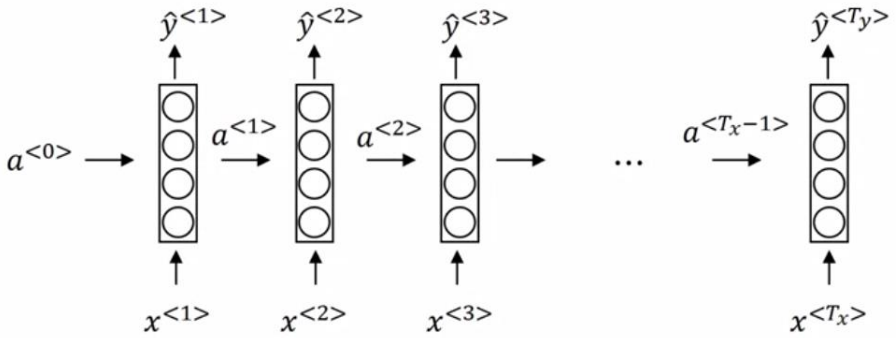
- Esta notação indica a posição temporal do elemento na sequencia
- $X^{(i)<t>}$: t-ésimo elemento da i-ésima sequencia (amostra)
- $T_x^{(i)}$: tamanho da i-ésima sequencia de entrada
- $T_y^{(i)}$: tamanho da i-ésima sequencia de saída

Redes Neurais Recorrentes: Notação



- Enquanto os pesos das conexões tradicionais são denotados por W_{ax}, os pesos da camada de contexto podem ser denotados por W_{aa}.
- Os pesos da camada de saída serão denotados por W_{ya}.
- **Os vetores de pesos são os mesmos ao longo da exibição da sequencia temporal**.

Redes Neurais Recorrentes: Forward Step



$$a^{<1>} = g(W_{aa} \cdot a^{<0>} + W_{ax} \cdot x^{<1>} + b_a)$$

$$\hat{y}^{<1>} = g(W_{ya} \cdot a^{<1>} + b_1)$$

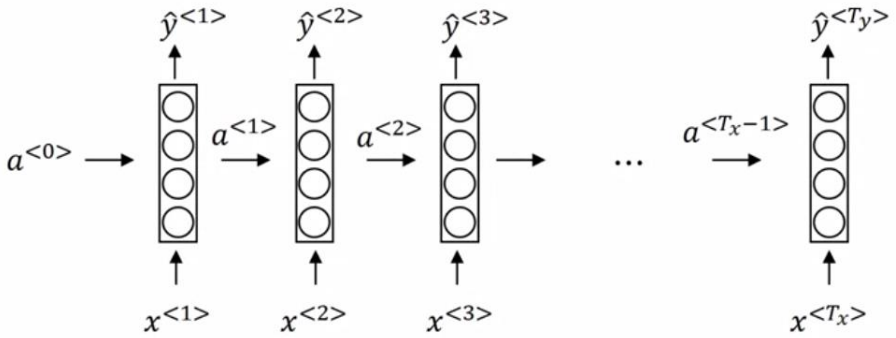
- W_{ax} → matriz de pesos que será multiplicado por um vetor X para gerar um vetor de ‘a’ (ativação).
- W_{ya} → matriz de pesos que será multiplicado por um vetor ‘a’ para gerar um vetor de ‘y’ (saídas).
- W_{aa} → matriz de pesos que será multiplicado por um vetor ‘a’ para gerar um vetor de ‘a’ (ativação).

Redes Neurais Recorrentes: Forward Step

Exemplo: Se o vocabulário tiver 10000 elementos e camada oculta igual a 100

$W_{aa} \rightarrow [100x100]$

$W_{ax} \rightarrow [100x10000]$



$$a^{<t>} = g(W_{aa} \cdot a^{<t-1>} + W_{ax} \cdot x^{<t>} + b_a)$$

$$\hat{y}^{<t>} = g(W_{ya} \cdot a^{<t>} + b_y)$$

$$a^{<t>} = g(W_a \cdot [a^{<t-1>}, x^{<t>}] + b_a)$$

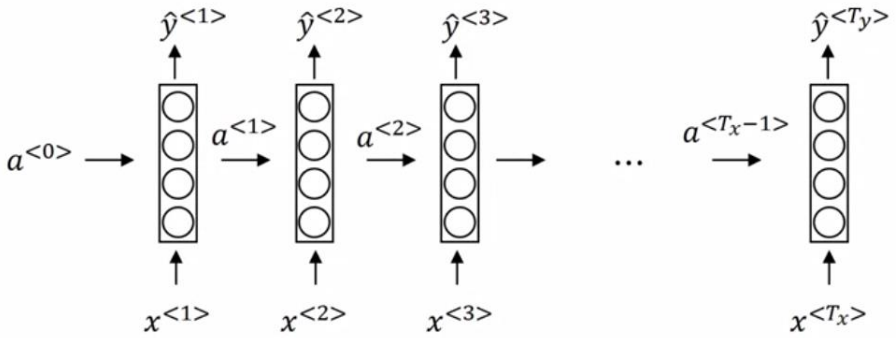
$$\hat{y}^{<t>} = g(W_y \cdot a^{<t>} + b_y)$$

Onde:

$$[W_{aa} \quad | \quad W_{ax}] = W_a$$

$$[a^{<t-1>}, x^{<t>}] \rightarrow \begin{bmatrix} a^{<t>} \\ - - \\ x^{(t)} \end{bmatrix}$$

Redes Neurais Recorrentes: Backward Step



- Pode ser utilizada a função de custo para entropia binária:

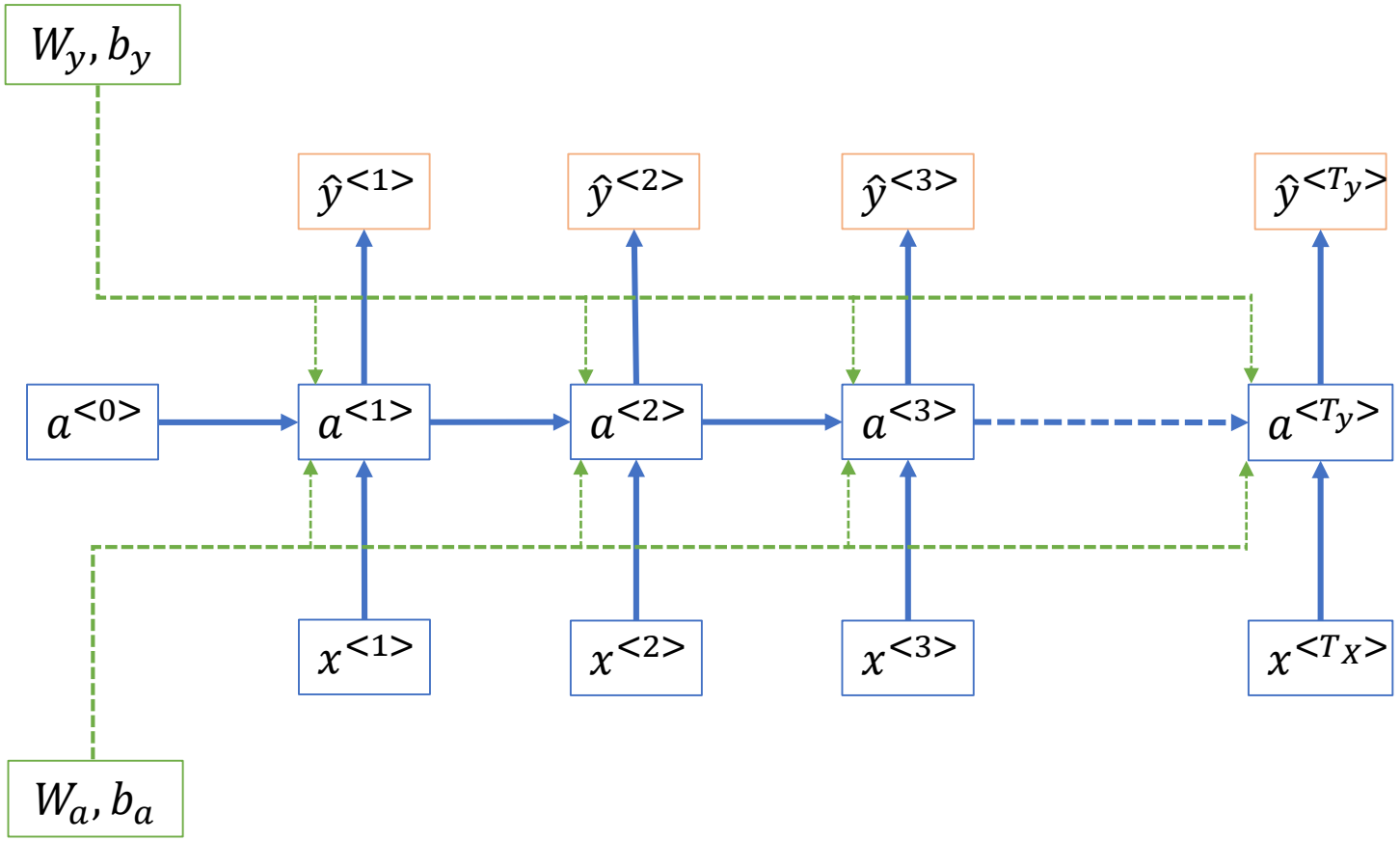
$$\mathcal{L}(\hat{y}^{<t>}, y^{<t>}) = -y^{<t>} \log \hat{y}^{<t>} - (1 - y^{<t>})\ log(1 - \hat{y}^{<t>})$$

- O custo da sequencia inteira pode ser dado por:

$$\mathcal{L}\{\hat{y}, y\} = \sum_{t=1}^{T_y} \mathcal{L}(\hat{y}^{<t>}, y^{<t>})$$

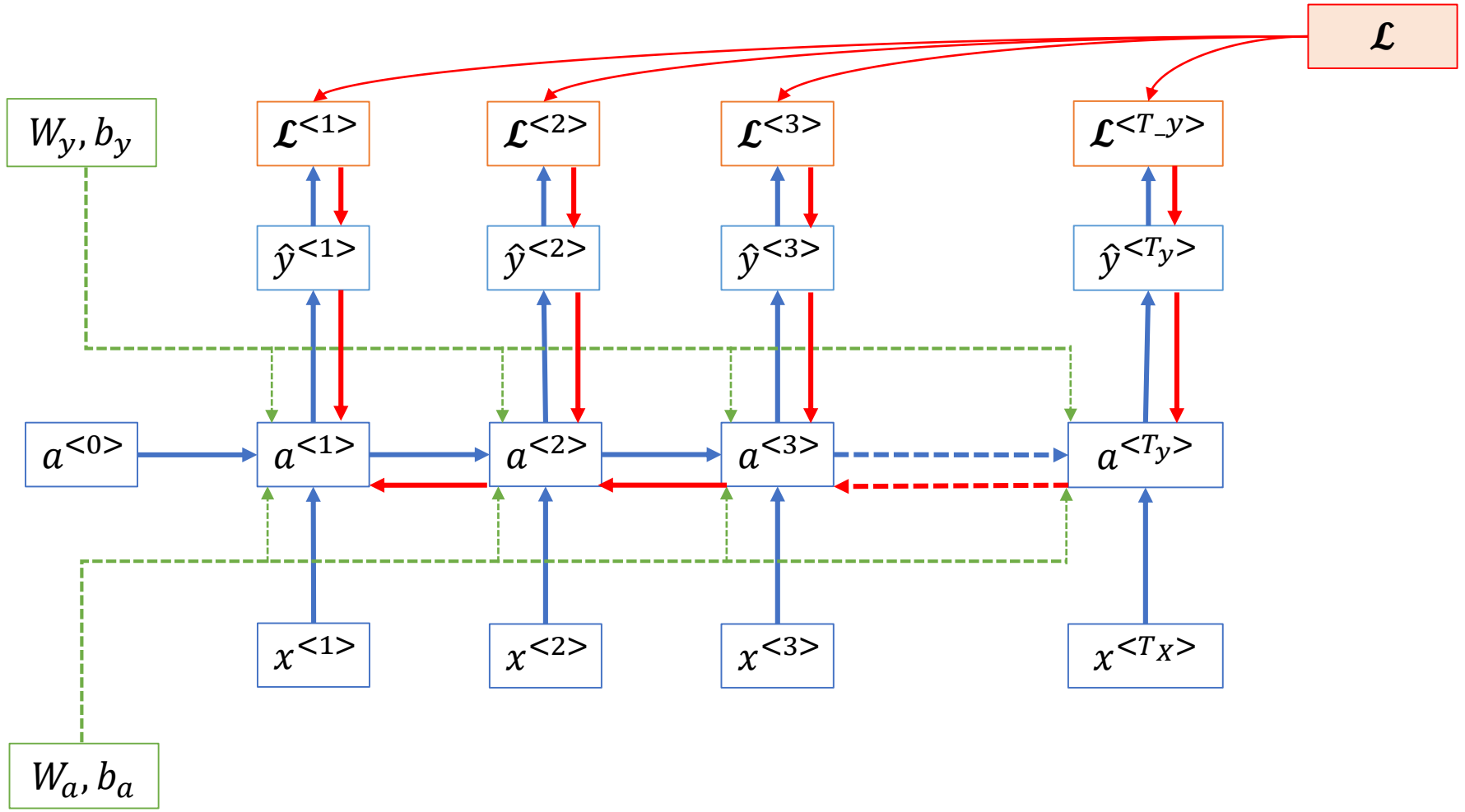
- ou seja, devem ser computados os erros para cada instante de tempo associado a cada elemento da sequencia e, a seguir, é calculado o erro geral associado a sequencia.

Redes Neurais Recorrentes: Backward Step

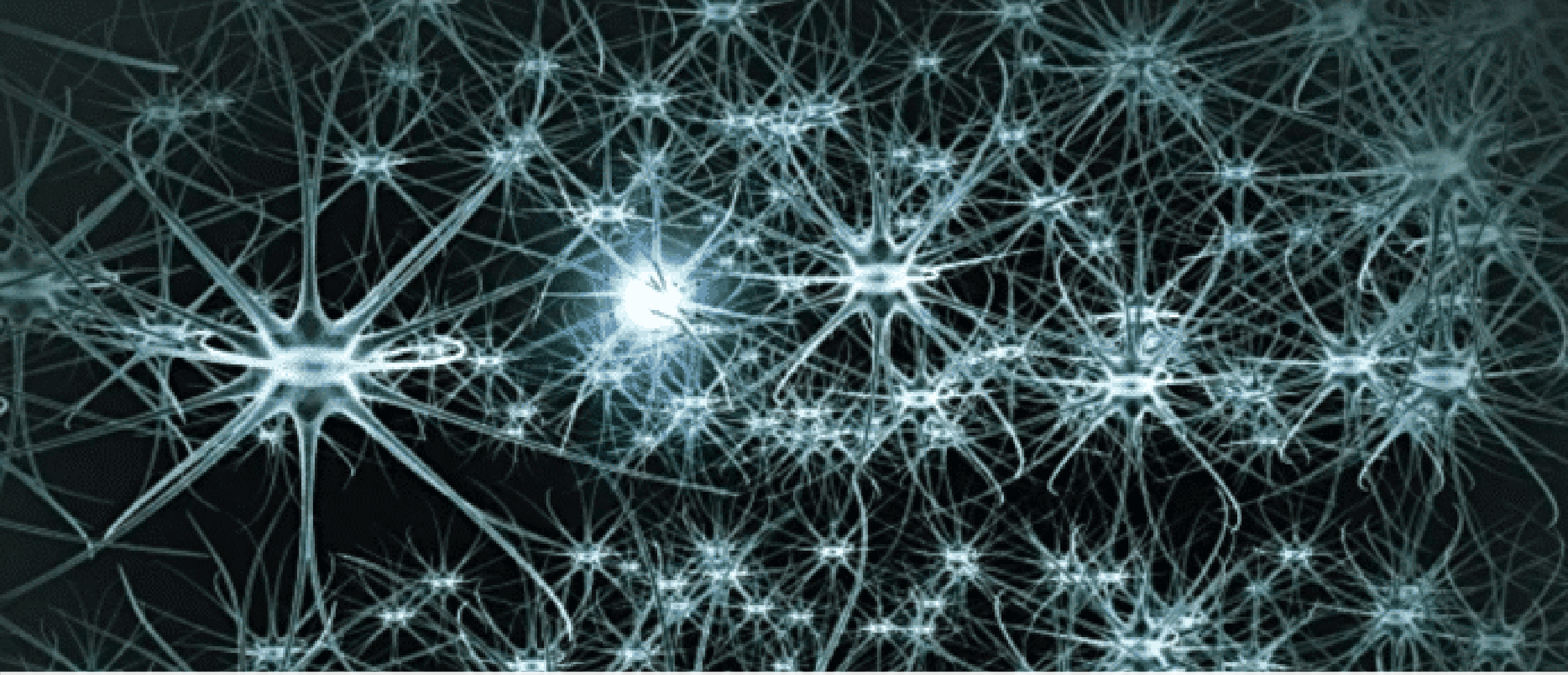


RNN: Backpropagation Thourgh Time (BPTT)

Se a rede ficar muito profunda, aparecerão os problemas de gradiente



https://www.mladdict.com/elman-recurrent-neural-network-simulator



RNN’s: Variantes Arquiteturais

Esta Foto de Autor Desconhecido está licenciado em CC BY-SA

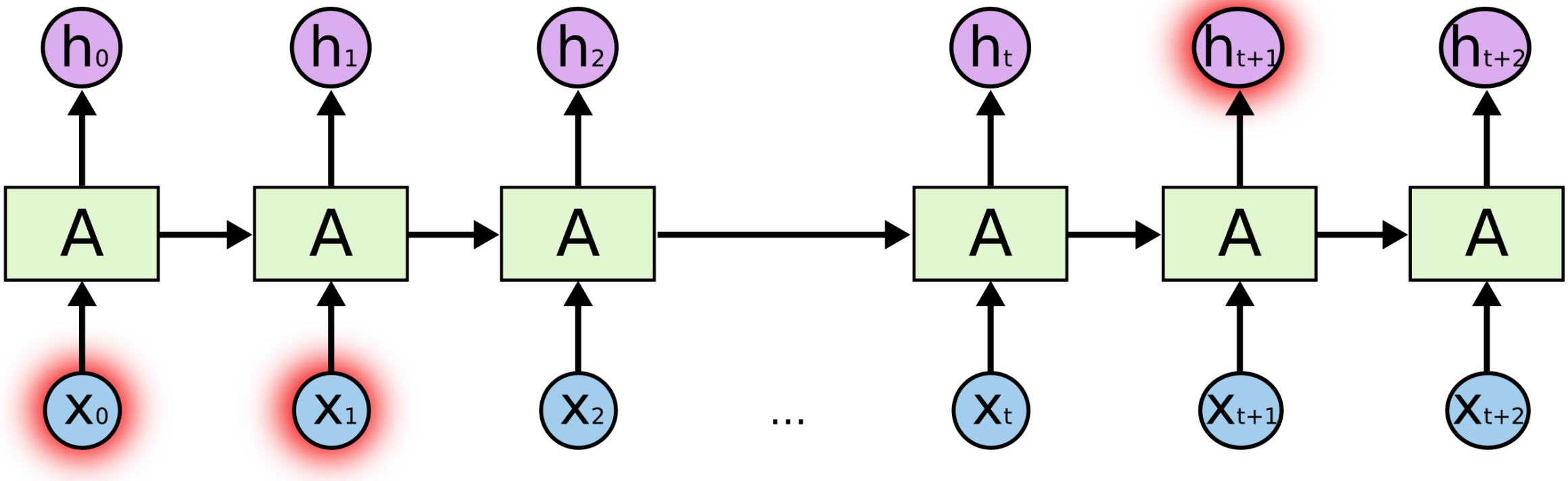
RNN’s: Variantes Arquiteturais



- Ao longo dos anos, os pesquisadores fizeram diversas propostas a fim de solucionar ou minimizar os problemas associados a redes profundas (Desaparecimento ou Explosão do Gradiente)

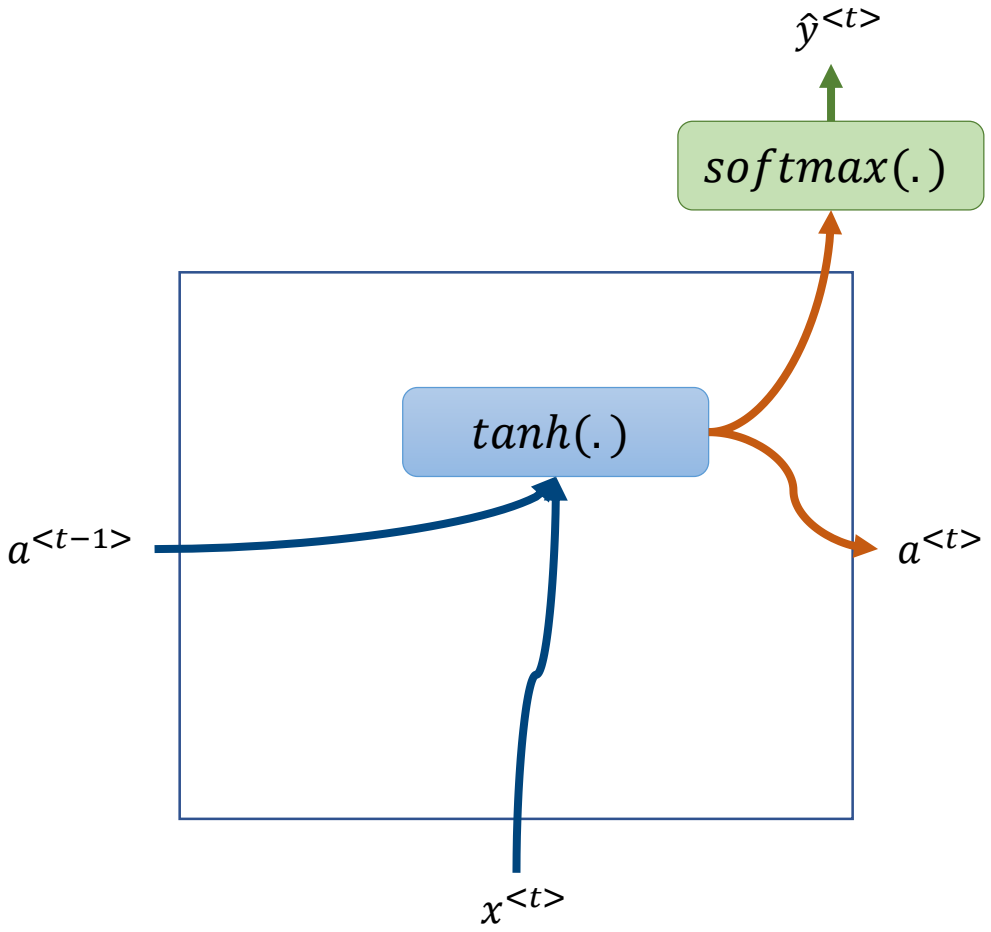
- No caso das Redes Recorrentes, as pesquisas geraram propostas de modificações no processamento da memória interna da rede, a fim de contornar os problemas e permitir outras aplicações destas redes.

RNN's: Variantes Arquiteturais



- Mesmo as RNN's tradicionais possuem dificuldade em associar elementos muito distantes no tempo (long-term dependencies)
- Exemplo: "The **cat**, which already ate..., **was** full"

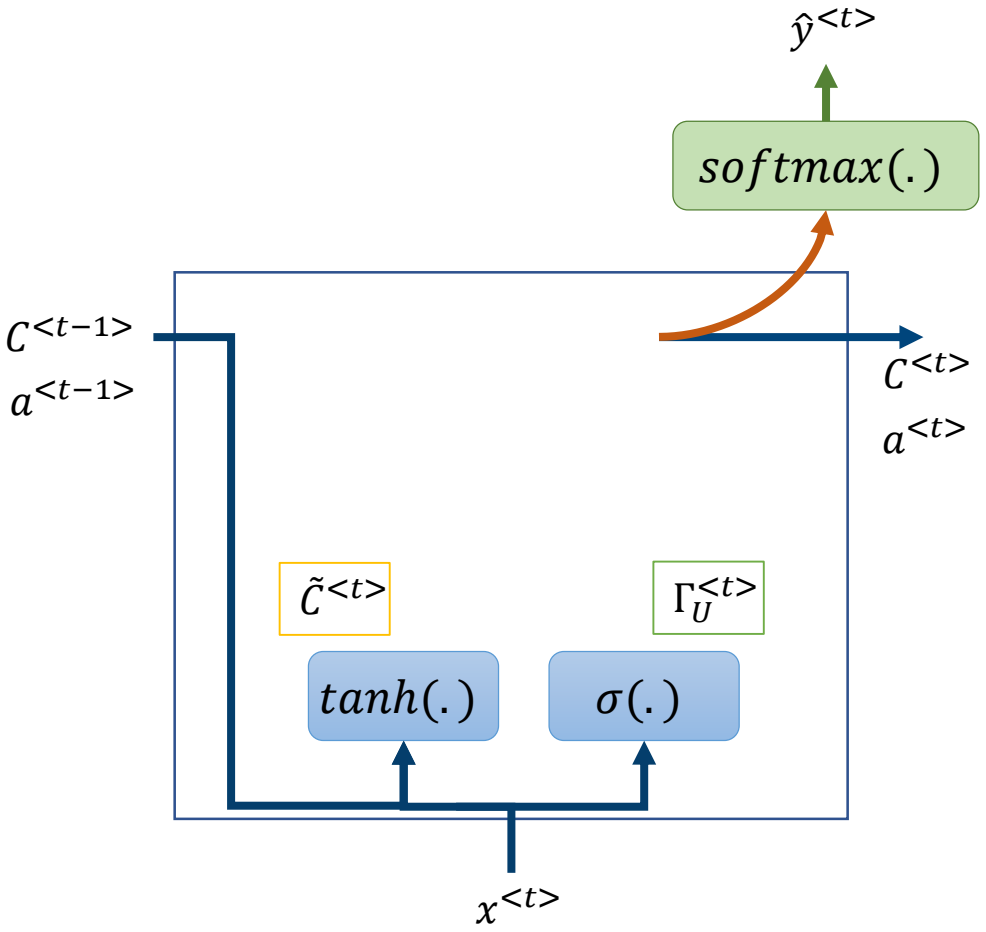
RNN's: Gated Recurrent Network (GRU)



- A figura ao lado mostra uma célula de uma RNN, ou seja um neurônio recorrente, para permitir a análise do fluxo de informações ao longo da rede.

- A partir do conceito de "Célula de Memória" e "Gate" será possível criar um mecanismo de controle da memória de contexto da célula recorrente.

RNN's: Gated Recurrent Network (GRU)



- "Célula de Memória":

$$C^{<t>} = a^{<t>}$$

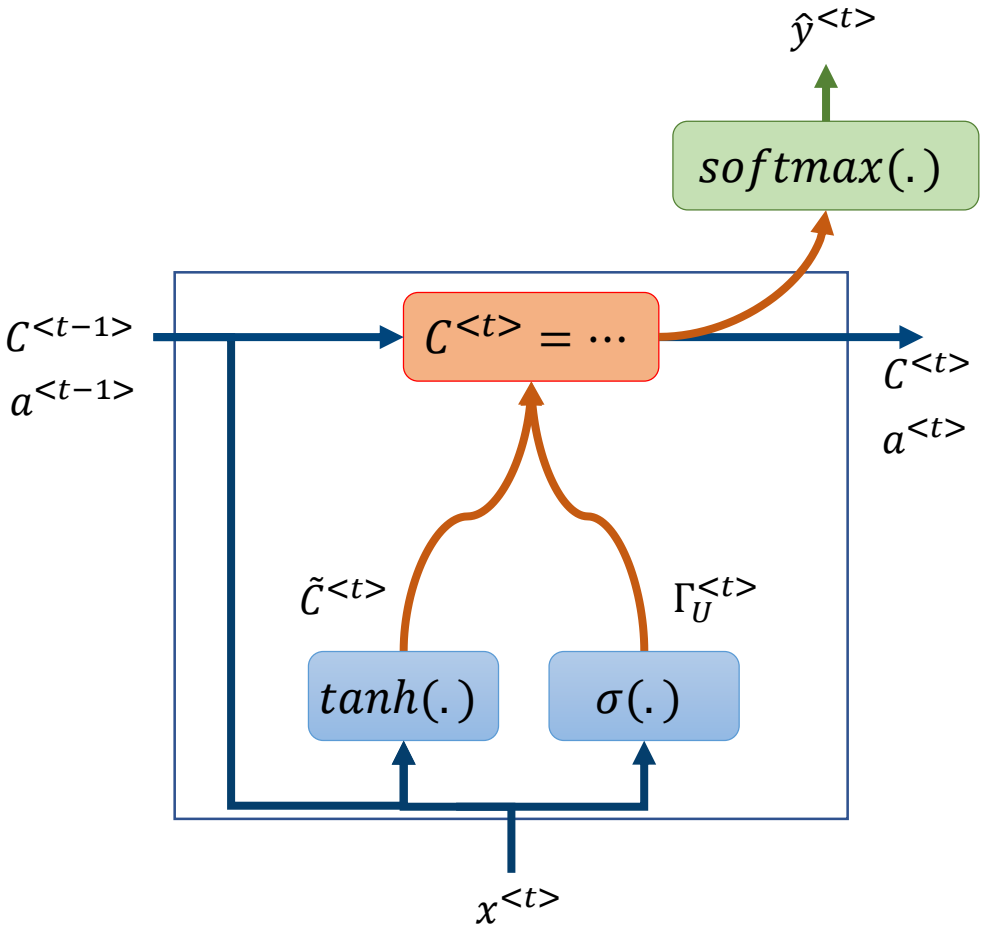
- A cada iteração, um valor novo é calculado pela célula:

$$\tilde{C}^{<t>} = \tanh(W_C[C^{<t-1>}, x^{<t>}] + b_C)$$

- O conceito de "**Gate**" é utilizado para decidir se o novo valor será ou não armazenado:

$$\Gamma_U = \sigma(W_U[C^{<t-1>}, x^{<t>}] + b_U)$$

RNN's: Gated Recurrent Network (GRU)



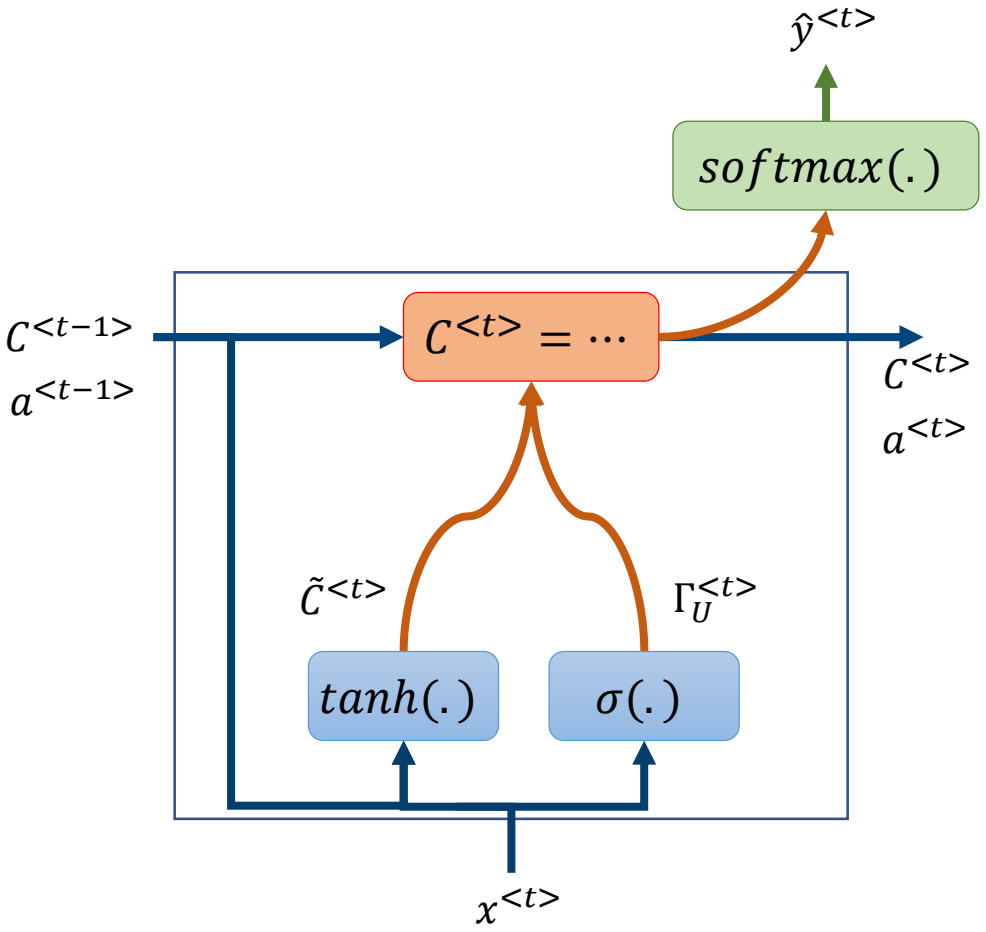
- Tipicamente o valor de Γ_U tenderá a saturar em "0" ou "1", permitindo que o valor de $\tilde{C}^{<t>}$ seja armazenado ou não na célula de memória.

- Desta forma, a célula de memória poderá manter uma informação "antiga" ao longo do tempo ou descarta-la quando não for mais necessária.

$$C^{<t>} = \Gamma_U * \tilde{C}^{<t>} + (1 - \Gamma_U) * C^{<t-1>}$$

- $\Gamma_U = 1$ → atualiza a memória
- $\Gamma_U = 0$ → mantém a memória do instante anterior

RNN's: Gated Recurrent Network (GRU)



- O modelo completo utiliza um Gate a mais para ponderação de **relevância** sobre o valor da memória, portanto além das equações já vistas:

$$\Gamma_U = \sigma(W_U[C^{<t-1>}, x^{<t>}] + b_U)$$

$$C^{<t>} = \Gamma_U * \tilde{C}^{<t>} + (1 - \Gamma_U) * C^{<t-1>}$$

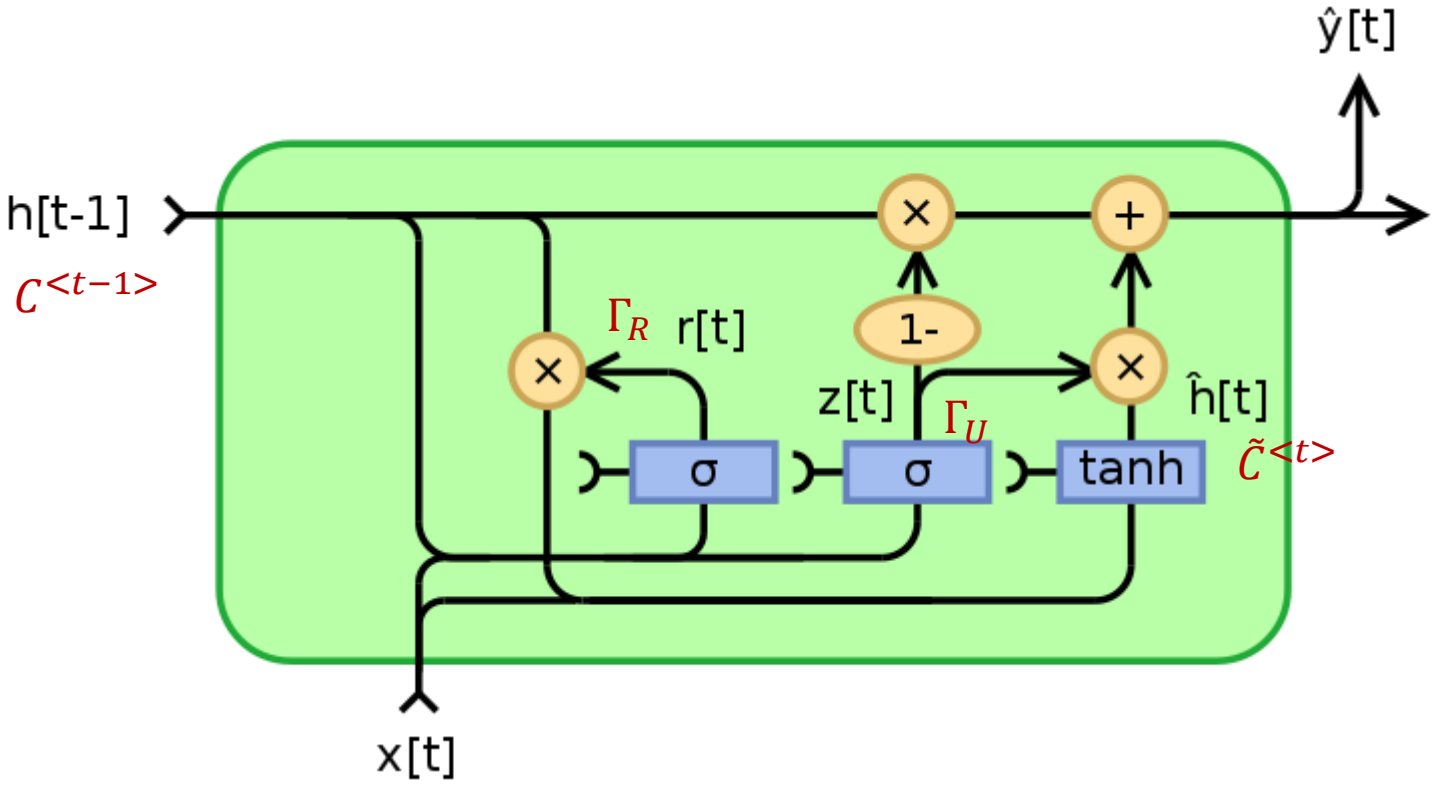
- Utiliza-se a equação:

$$\Gamma_R = \sigma\left(W_R\left[[C^{<t-1>}, x^{<t>}] + b_C\right]\right)$$

- Desta forma, a equação da "memória candidata" modifica-se para:

$$\tilde{C}^{<t>} = \tanh(W_C[\Gamma_R * C^{<t-1>}, x^{<t>}] + b_C)$$

RNN's: Gated Recurrent Network (GRU)



$$\Gamma_U = \sigma(W_U[C^{<t-1>}, x^{<t>}] + b_U)$$

$$C^{<t>} = \Gamma_U * \tilde{C}^{<t>} + (1 - \Gamma_U) * C^{<t-1>}$$

$$\Gamma_R = \sigma\left(W_R\left[[C^{<t-1>}, x^{<t>}] + b_C\right]\right)$$

$$\tilde{C}^{<t>} = \tanh(W_C[\Gamma_R * C^{<t-1>}, x^{<t>}] + b_C)$$

Fonte: https://en.wikipedia.org/wiki/File:Gated_Recurrent_Unit,_base_type.svg

RNN's: Long Short Term Memories (LSTM's)



forget valve
Γ_F

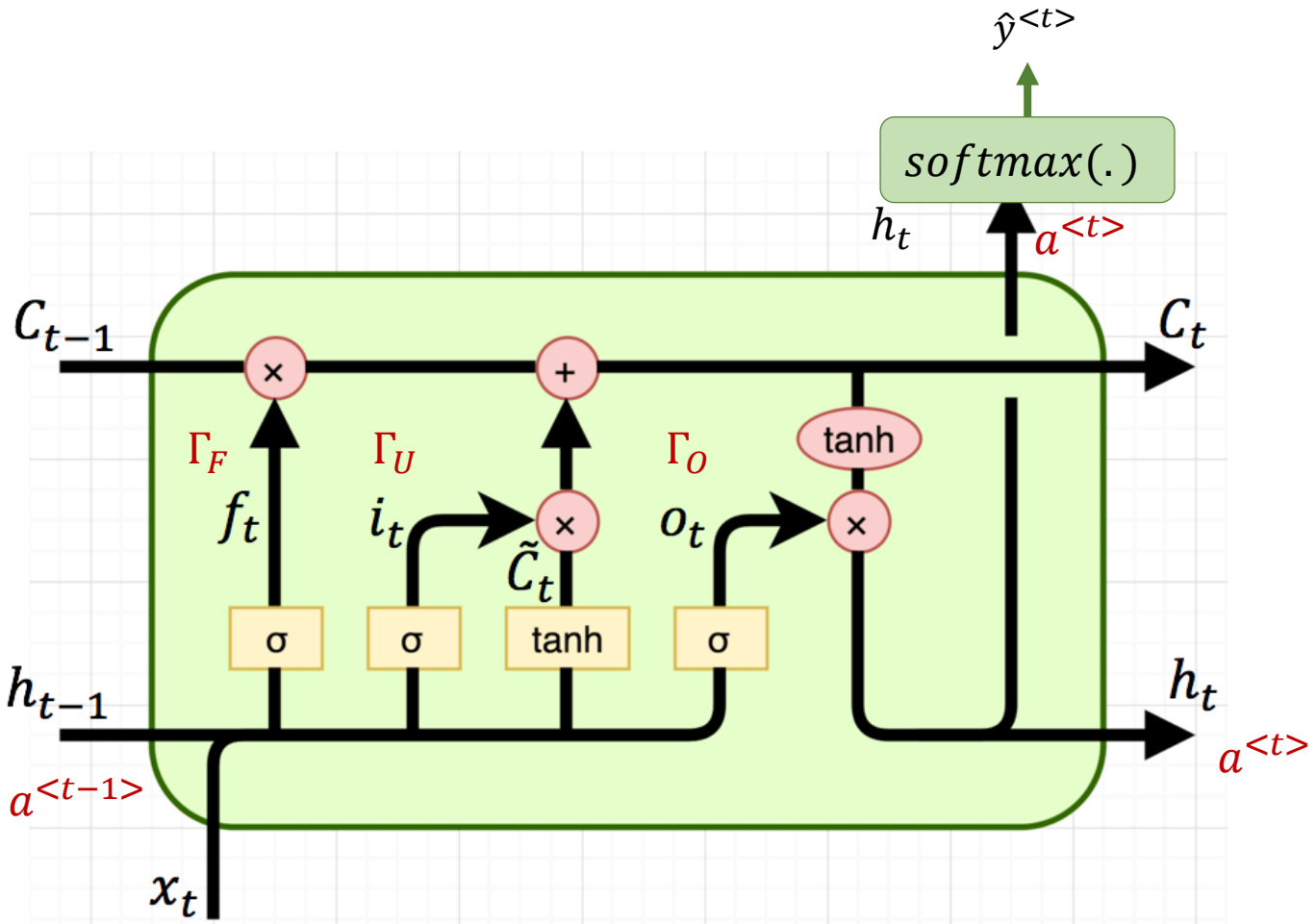


new memory valve
Γ_U

- As LSTM's compõem um tipo especial de Rede Recorrente, cuja célula de memória é mais complexa do que a GRU (embora a LSTM seja mais antiga – 1997)
- Suas principais diferenças em relação à GRU, é a inclusão de um sistema para "**esquecer**" memórias antigas e atualização de **memórias novas** utilizando gates distintos ao invés de um único gate Γ_U
- Também apresenta um gate de saída, Γ_O, que irá "**filtrar**" o conteúdo que será disponibilizado como a saída da célula, levando-se em conta a memória de longo prazo e a de curto prazo.

Fonte: https://medium.com/mlreview/understanding-lstm-and-its-diagrams-37e2f46f1714

RNN's: Long Short Term Memories (LSTM's)



Equações do LSTM:

$$\tilde{C}^{<t>} = \tanh(W_C[a^{<t-1>}, x^{<t>}] + b_C)$$

$$\Gamma_U = \sigma(W_U[a^{<t-1>}, x^{<t>}] + b_U)$$

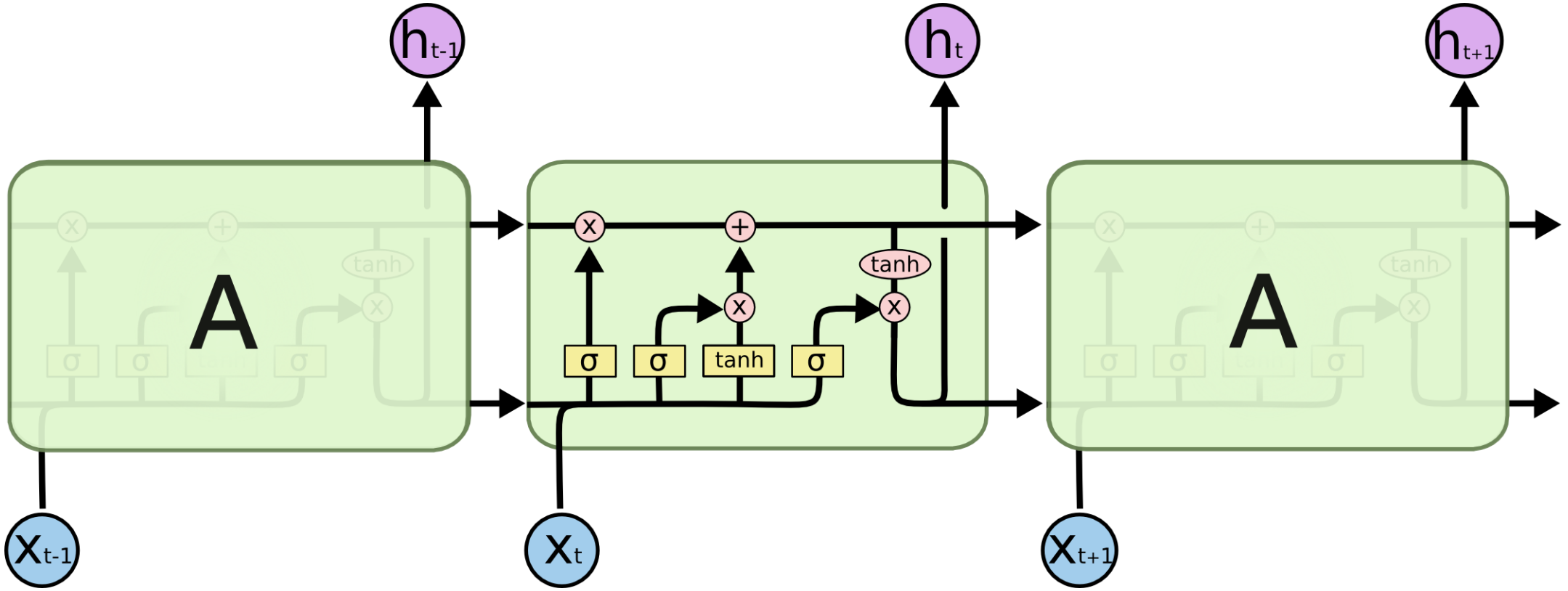
$$\Gamma_F = \sigma(W_F[a^{<t-1>}, x^{<t>}] + b_F)$$

$$\Gamma_O = \sigma(W_O[a^{<t-1>}, x^{<t>}] + b_O)$$

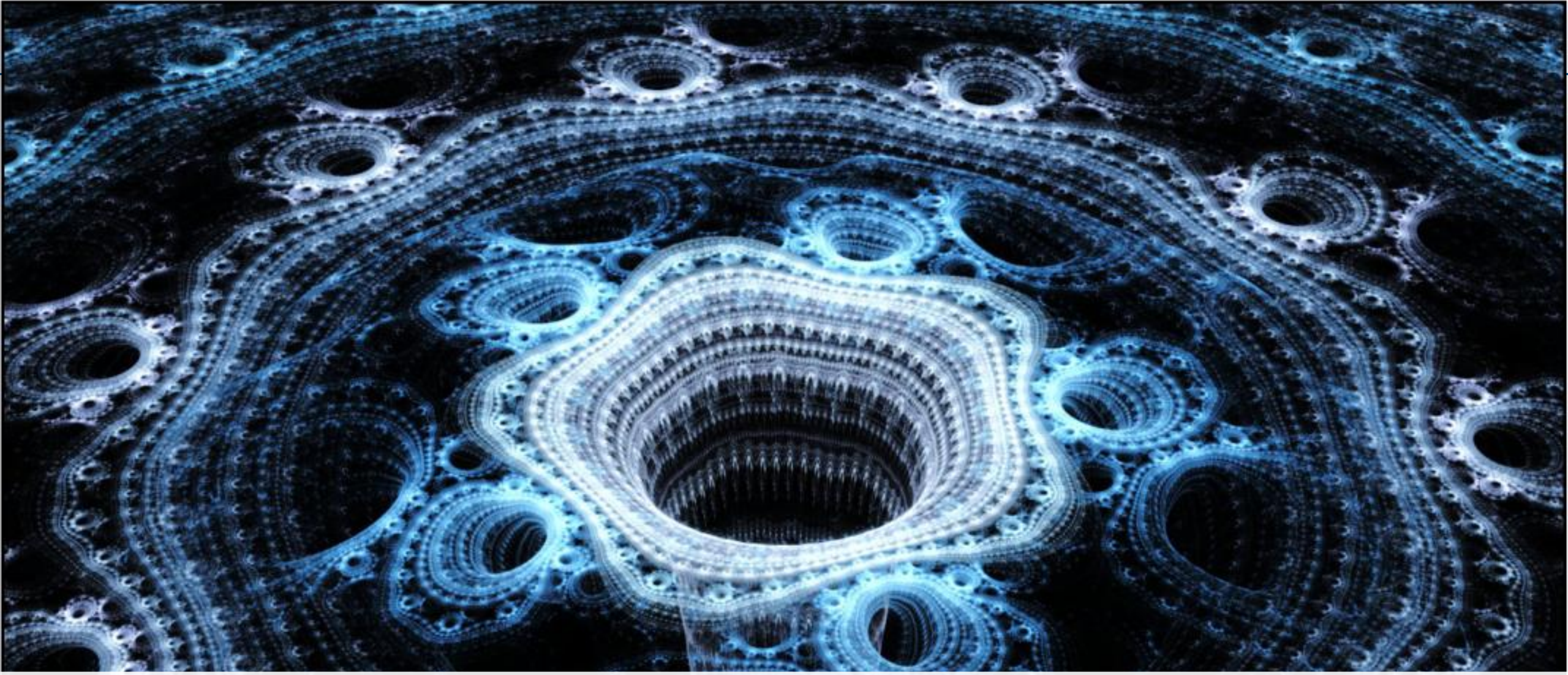
$$C^{<t>} = \Gamma_U * \tilde{C}^{<t>} + \Gamma_F * C^{<t-1>}$$

$$a^{<t>} = \Gamma_O * \tanh(C^{<t>})$$

RNN's: Long Short Term Memories (LSTM's)



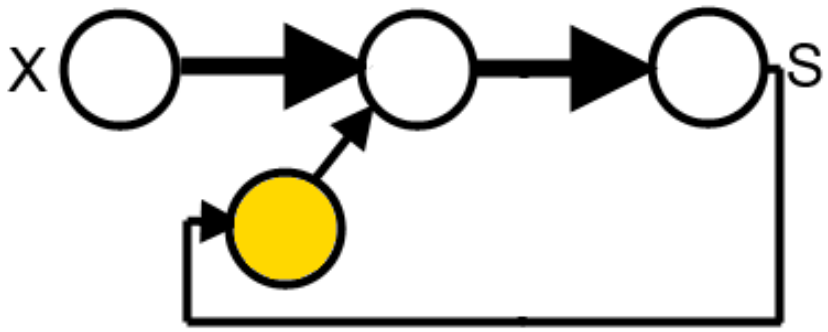
Esta Foto de Autor Desconhecido está licenciado em CC BY-SA



RNNs: Sequências Temporais

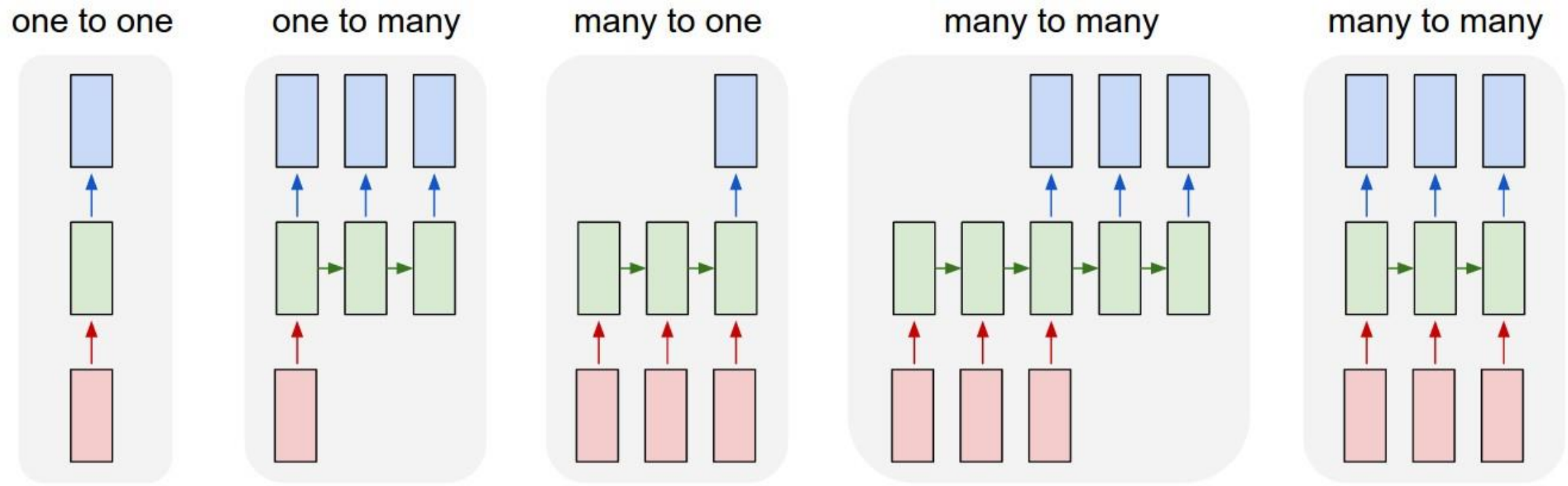
Fonte: http://cdn.jornalgrandebahia.com.br/2018/01/Intelig%C3%AAncia-Artificial-IA-106-500x333.jpg

RNN's: Sequencias Temporais



- As arquiteturas tradicionais de redes (MLP e CNN) normalmente recebem vetores de tamanho FIXO em suas entradas.
- As redes recorrentes possuem aplicações variadas, normalmente associadas a processamento de sequencias temporais. Desta forma, suas entradas podem ser visualizadas como "variáveis ao longo do tempo"

RNN's: Sequencias Temporais



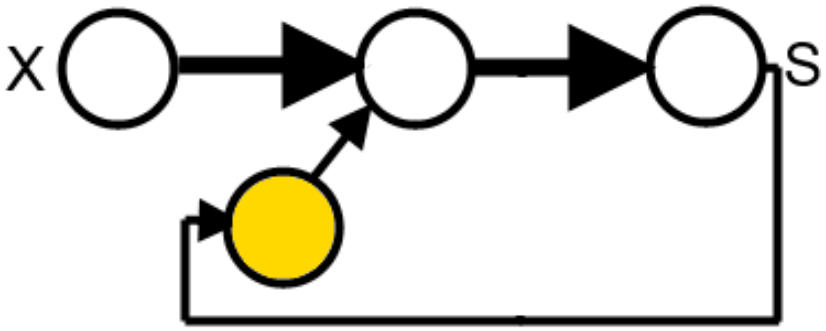
$x = elemento, y = sequencia$
Síntese (Texto/Música)

$x = sequencia, y = 0/1$
Análise de Sentimento

$x = elemento, y = sequencia$

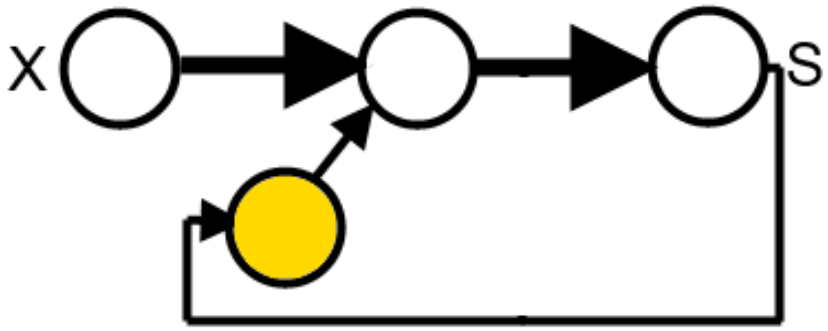
Fonte: http://karpathy.github.io/assets/rnn/diags.jpeg

RNN's: Sequencias Temporais



- Como representar palavras individuais numa sentença?
- Modelos tradicionais não funcionam direito, uma vez que elas não possuem memória capaz de relacionar os posicionamentos dos textos ao longo das sentenças.
- De certa forma é análogo ao processo das Redes Convolucionais onde espera-se que o conhecimento adquirido para uma parte de uma imagem seja generalizado para outras partes da imagem.

RNN's: Word Embeddings

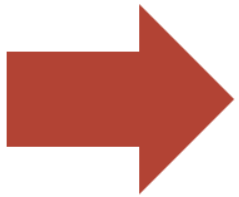


I	want	a	glass	of	orange	_____.
4343	9665	1	3852	6163	6257	

- Como representar palavras individuais numa sentença?
- Temos que usar um Vocabulário (Dicionário)
- É possível codificar cada palavra através de 1 hot encoding, de modo que neste exemplo, cada palavra se torna um vetor de N posições.
- Desta forma, o problema se torna um problema de aprendizado supervisionado onde os vetores X são usados para identificar a saída Y.

RNN's: Word Embeddings

Vocabulary:
Man, woman, boy, girl, prince, princess, queen, king, monarch



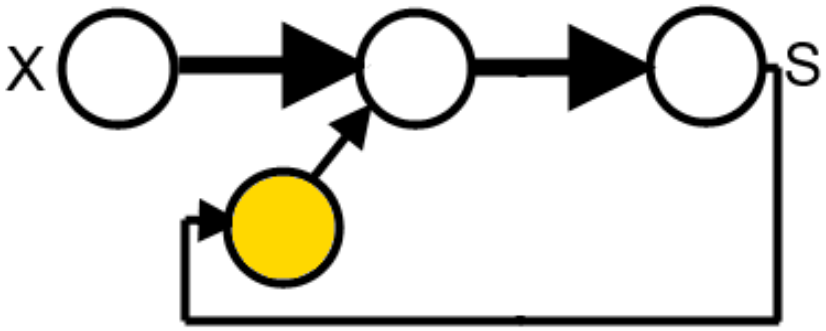
	1	2	3	4	5	6	7	8	9
man	1	0	0	0	0	0	0	0	0
woman	0	1	0	0	0	0	0	0	0
boy	0	0	1	0	0	0	0	0	0
girl	0	0	0	1	0	0	0	0	0
prince	0	0	0	0	1	0	0	0	0
princess	0	0	0	0	0	1	0	0	0
queen	0	0	0	0	0	0	1	0	0
king	0	0	0	0	0	0	0	1	0
monarch	0	0	0	0	0	0	0	0	1

Each word gets a 1x9 vector representation

- O exemplo é pequeno, aplicações comerciais podem usar de 30 mil a 50 mil, ou até 100 mil palavras
- Palavras desconhecidas podem ser associadas a um Token UNK

Fonte: https://shanelynnwebsite-mid9n9g1q9y8tt.netdna-ssl.com/wp-content/uploads/2018/01/one-hot-word-embedding-vectors.png

RNN's: Skip-Gram



- O modelo conhecido como Skip-Gram é bastante utilizado em NLP e sua técnica é bastante simples
- A idéia central é utilizar uma arquitetura Neural para treinar e se adaptar a um determinado vocabulário e utilizar os pesos desta rede treinada como uma "representação de features" do próprio vocabulário em outras tarefas de sequenciamento.

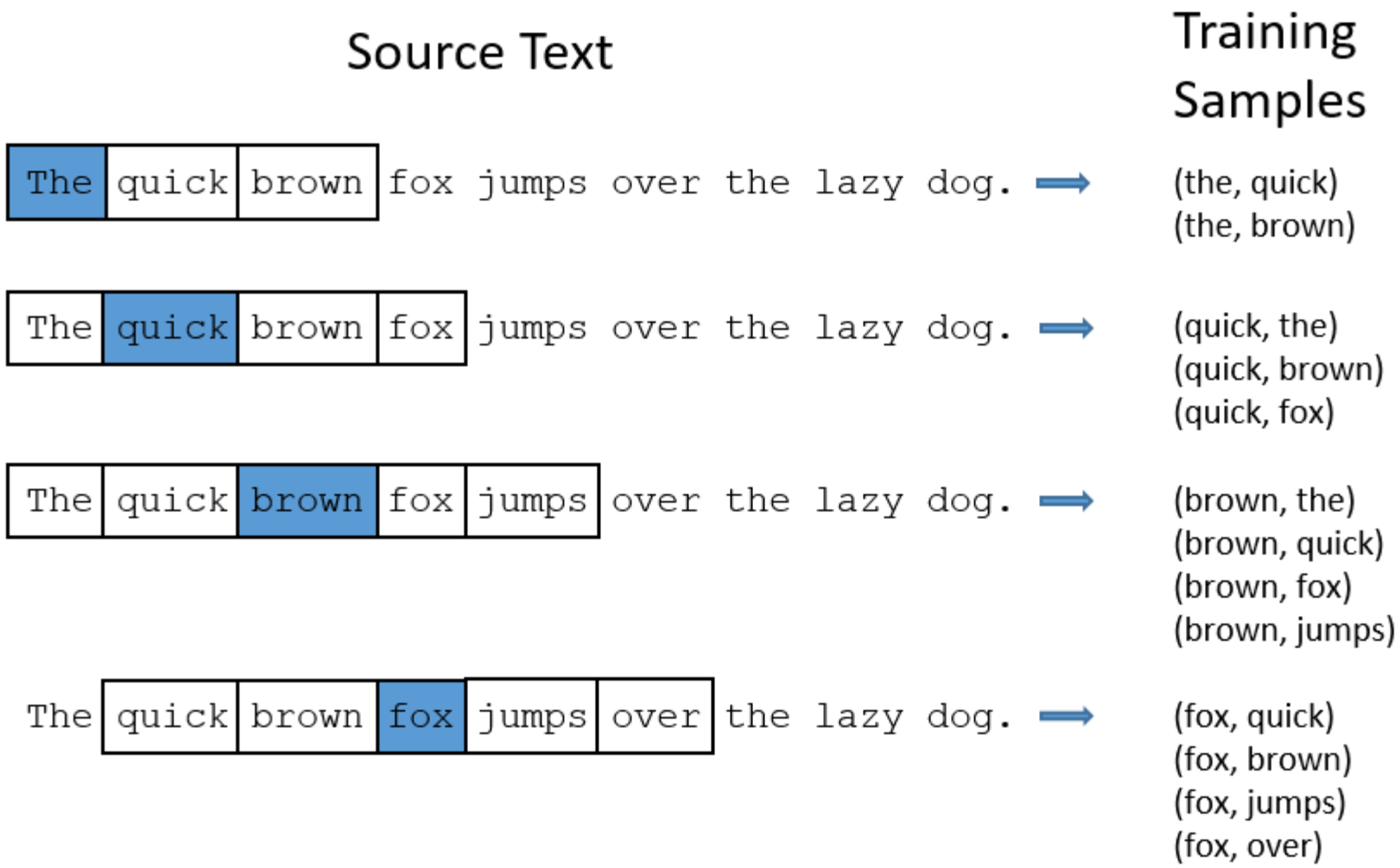
Esta Foto de Autor Desconhecido está licenciado em CC BY-NC

RNN's: Skip-Gram

Aqui foi utilizada uma janela de 2 unidades

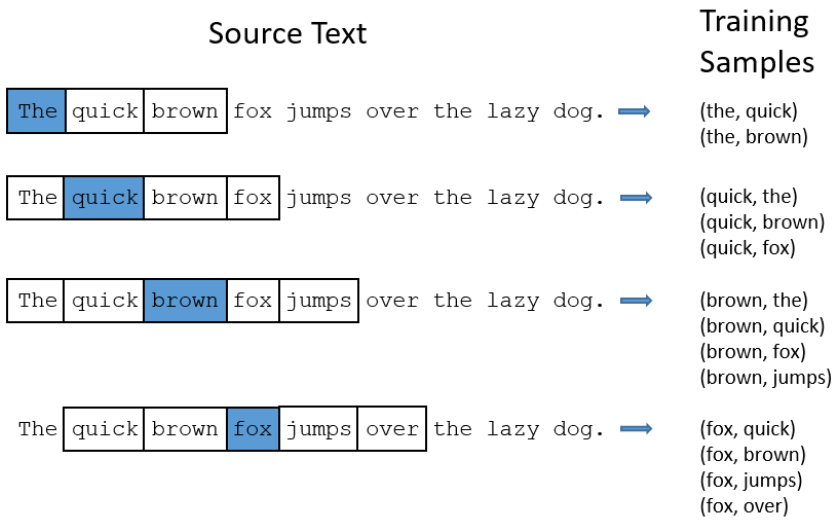
O termo em destaque é a amostra de entrada

Os elementos da janela serão as saídas



Esta Foto de Autor Desconhecido está licenciado em CC BY-NC

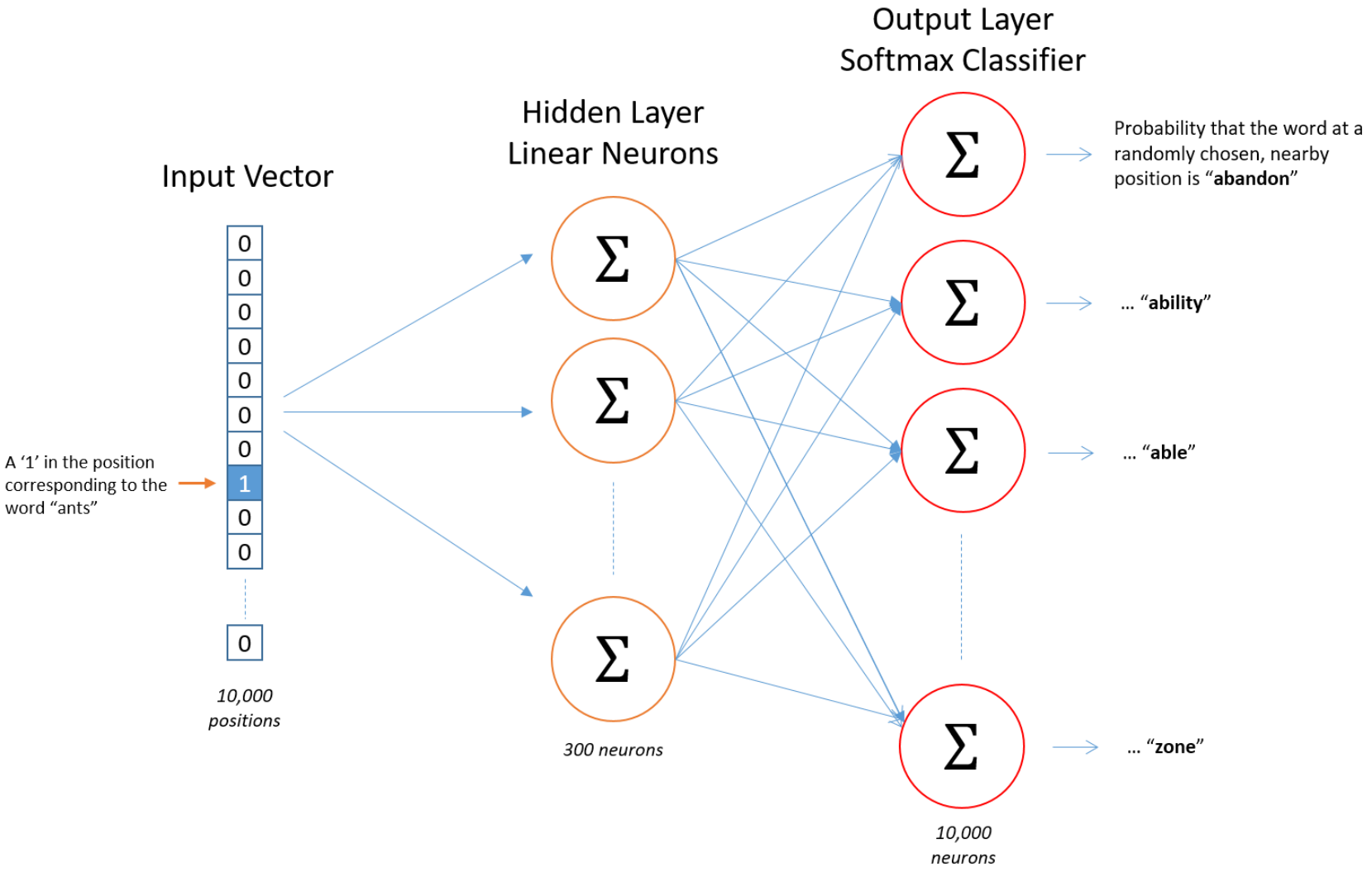
RNN's: Skip-Gram



- Como já foi visto, as palavras devem ser convertidas em vetores apropriados para serem inseridas em uma Rede Neural e uma das formas de se fazer isso é o One-hot encoding.
- Desta forma, se o vocabulário for composto por 10.000 palavras, cada vetor de entrada terá 10.000 componentes.
- A arquitetura neural de treinamento deverá determinar a probabilidade de cada saída a partir de uma dada palavra de entrada.

Esta Foto de Autor Desconhecido está licenciado em CC BY-NC

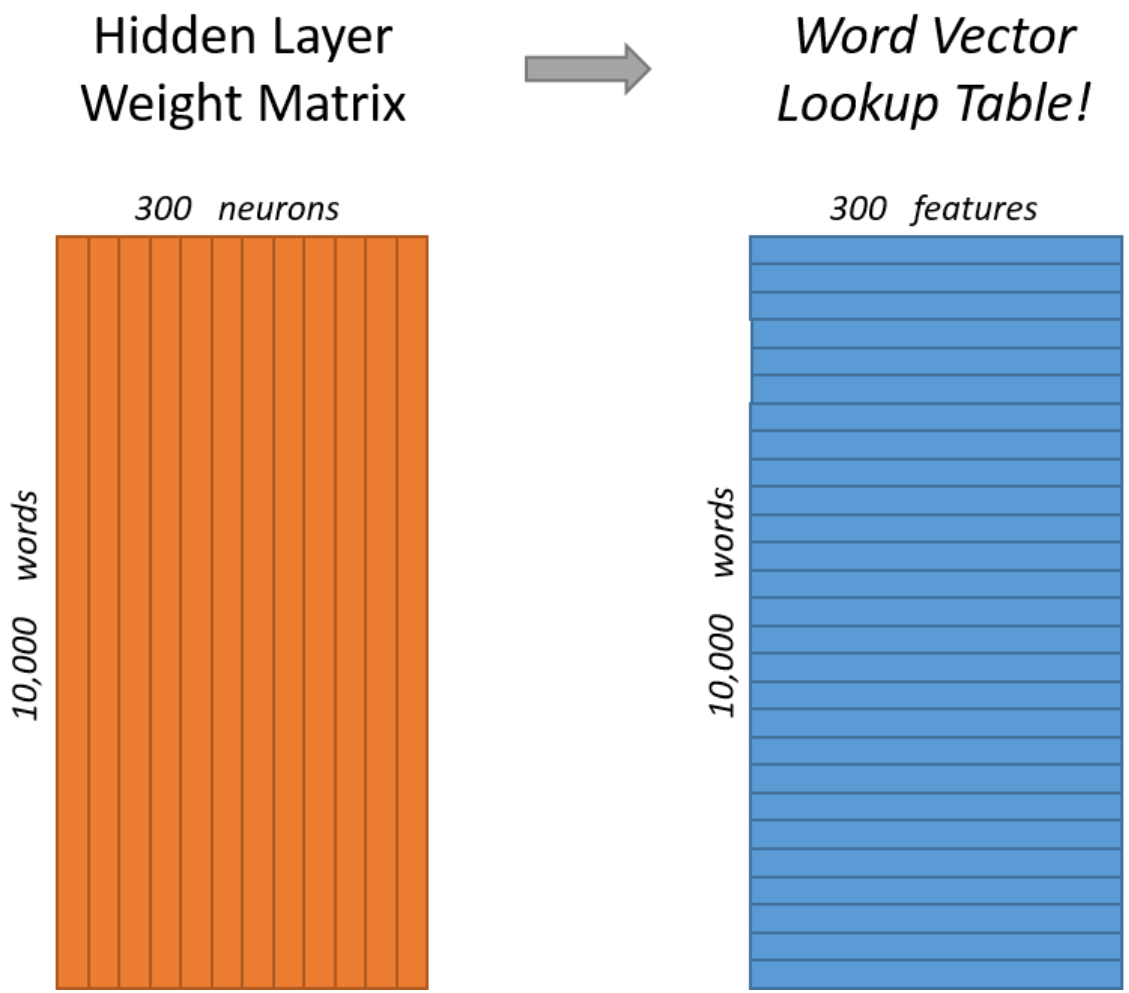
RNN's: Skip-Gram/Word2Vec



- Nesta rede, tanto a entrada quanto a saída são vetores de 10.000 posições (one-hot encoded).
- A camada oculta não possui função de ativação
- Após o treinamento, a saída será uma distribuição de probabilidades

Fonte:http://mccormickml.com/assets/word2vec/skip_gram_net_arch.png

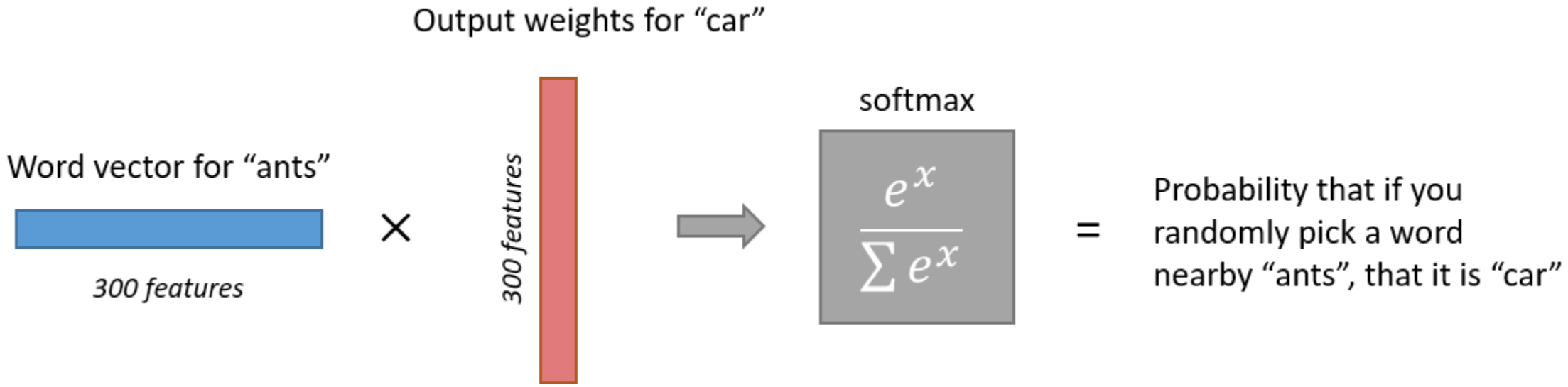
RNN's: Skip-Gram/Word2Vec



- Após a realização do treinamento, os pesos da camada oculta estarão ajustados para serem "excitados" ou "inibidos" em função de determinadas palavras
- Palavras semanticamente próximas ou com contexto próximo, terão vetores também próximos. O contexto é dado pelas palavras da vizinhança, determinada pela janela.
- Os pesos ajustados podem ser usados como features de cada palavra do vocabulário

Fonte:http://mccormickml.com/assets/word2vec/skip_gram_net_arch.png

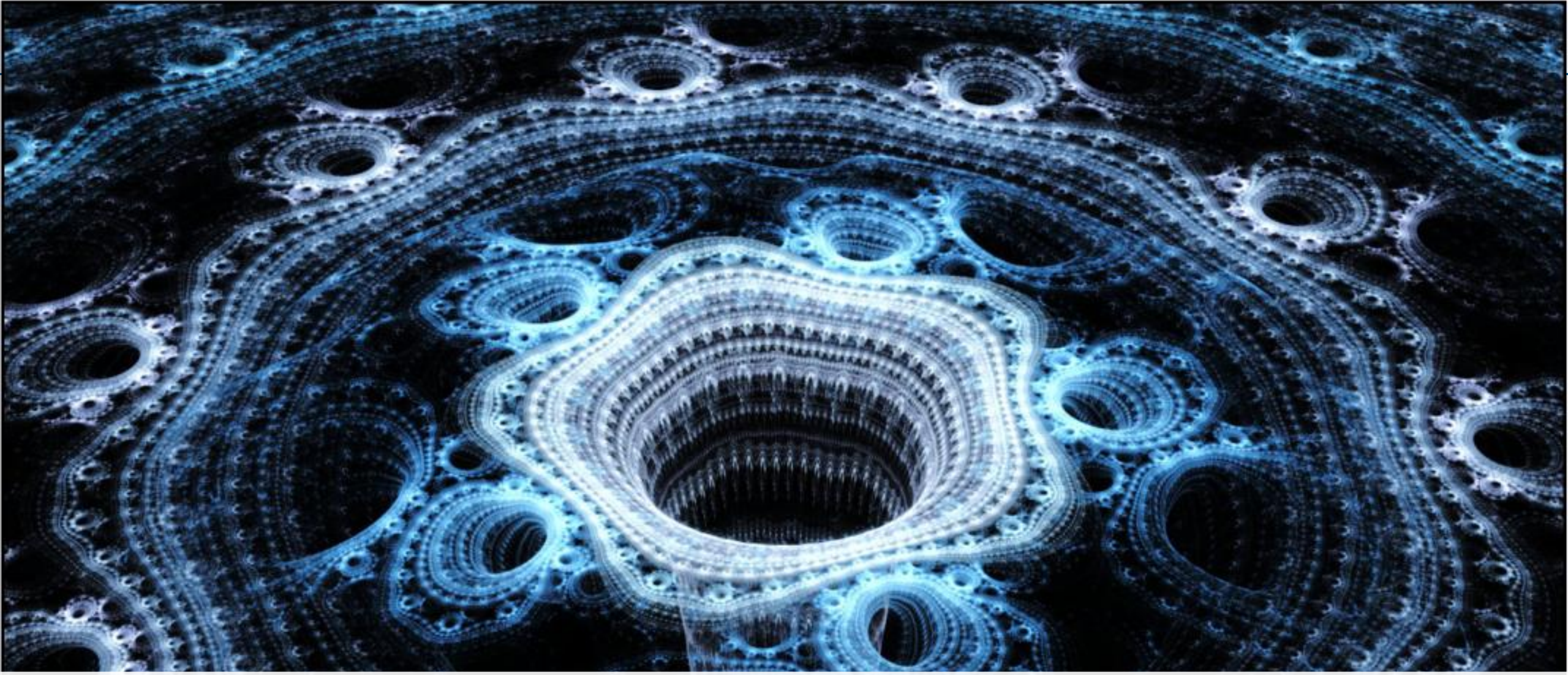
RNN's: Skip-Gram/Word2Vec



- Agora, cada linha da matriz de pesos pode ser vista como uma 'lookup-table', uma vez que cada palavra (token) em sua representação one-hot encoding, irá ativar uma linha específica da matriz de pesos.

$$[0 \quad 0 \quad 0 \quad 1 \quad 0] \times \begin{bmatrix} 17 & 24 & 1 \\ 23 & 5 & 7 \\ 4 & 6 & 13 \\ 10 & 12 & 19 \\ 11 & 18 & 25 \end{bmatrix} = [10 \quad 12 \quad 19]$$

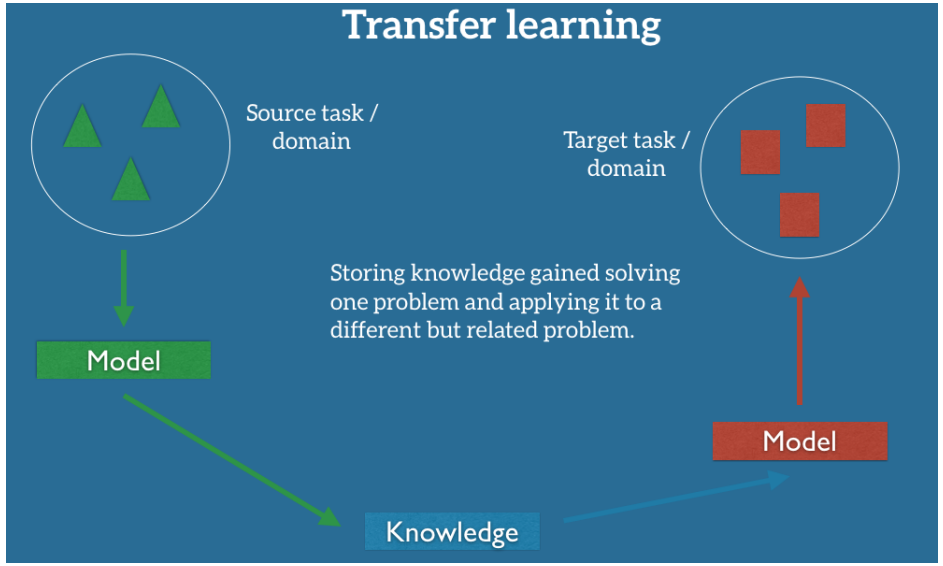
Fonte: http://mccormickml.com/assets/word2vec/output_weights_function.png



Deep Learning: Outras técnicas

Fonte: http://cdn.jornalgrandebahia.com.br/2018/01/Intelig%C3%AAncia-Artificial-IA-106-500x333.jpg

Deep Learning: Learning Transfer



- Learning Transfer é uma técnica utilizada em Aprendizado de Máquina, onde a base de conhecimento adquirida em um determinado problema passa a ser aplicada em outros problemas similares.

- Exemplo: a partir de uma rede treinada para reconhecer carros, utilizam-se os parâmetros do modelo para otimizar o aprendizado de imagens de caminhões.

Foto: http://ruder.io/transfer-learning/

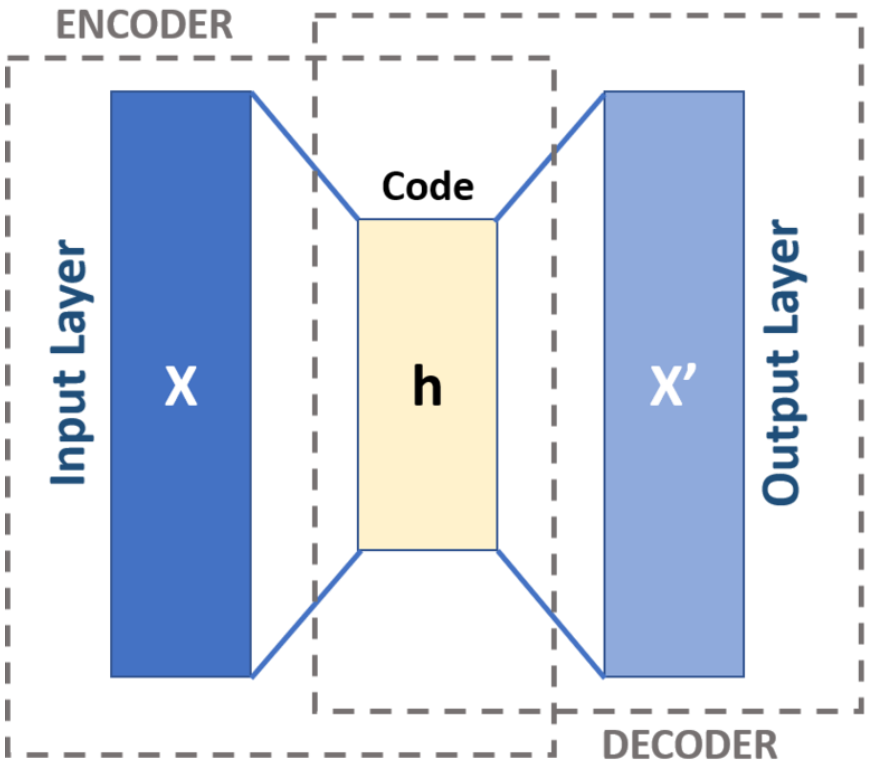
Deep Learning: Style Transfer



- Style Transfer é uma técnica utilizada em Deep Learning, a fim de transferir ou sobrepor uma imagem, definida como “estilo”, em outra imagem definida como destino, preservando seus aspectos principais.
- A idéia em que se baseia esta técnica é a utilização de uma função custo que “mistura” tanto a imagem-alvo quanto a imagem-estilo.
- Desta forma, o algoritmo de aprendizado tentará minimizar o erro associado à combinação das imagens.

Fonte: https://www.kaggle.com/basu369victor/style-transfer-deep-learning-algorithm

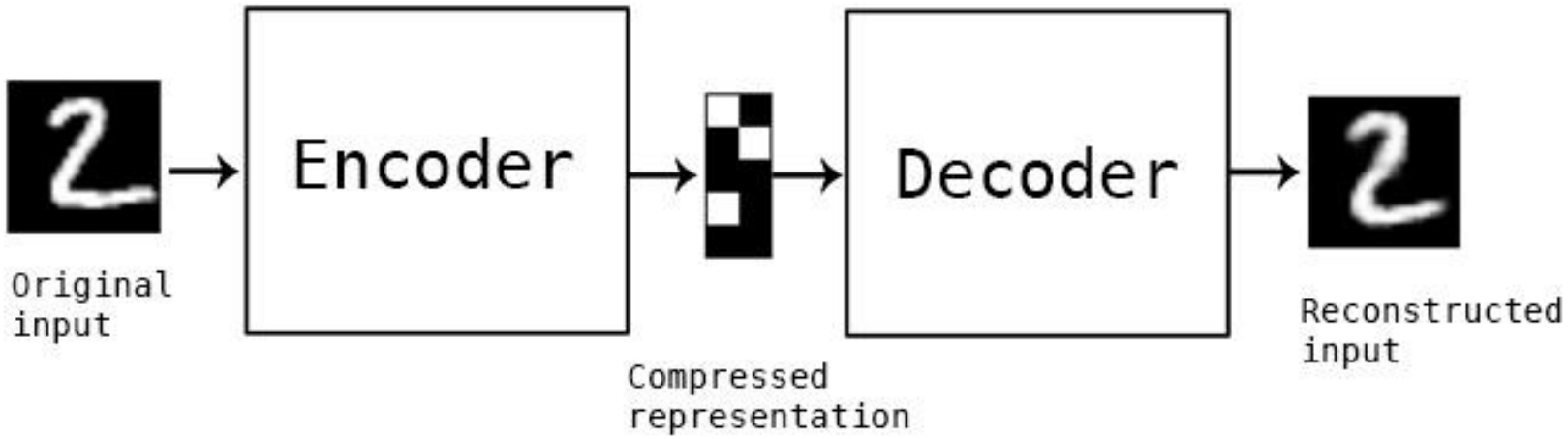
Deep Learning: Auto Encoder



- Um autoencoder é uma Rede Neural Artificial treinada para aprender codificações eficientes (representações) de um conjunto de dados.
- O treinamento é do tipo não-supervisionado.
- O principal interesse reside no fato de se obter uma representação reduzida (comprimida) dos dados originais, porém com a possibilidade de obtê-los a partir da reconstrução (decoder)

Fonte: https://upload.wikimedia.org/wikipedia/commons/thumb/3/37/Autoencoder_schema.png/800px-Autoencoder_schema.png

Deep Learning: Auto Encoder



- Modelo:

$$X = (X * W_1 + b_1) * W_2 + b_2$$

Fonte: https://blog.keras.io/img/ae/autoencoder_schema.jpg

Deep Learning: GAN's

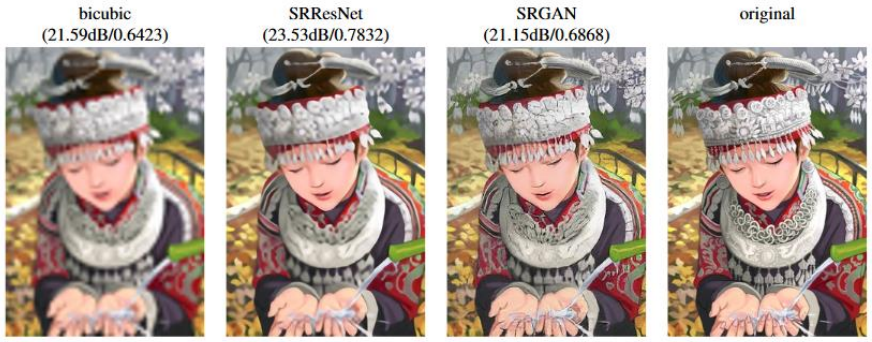


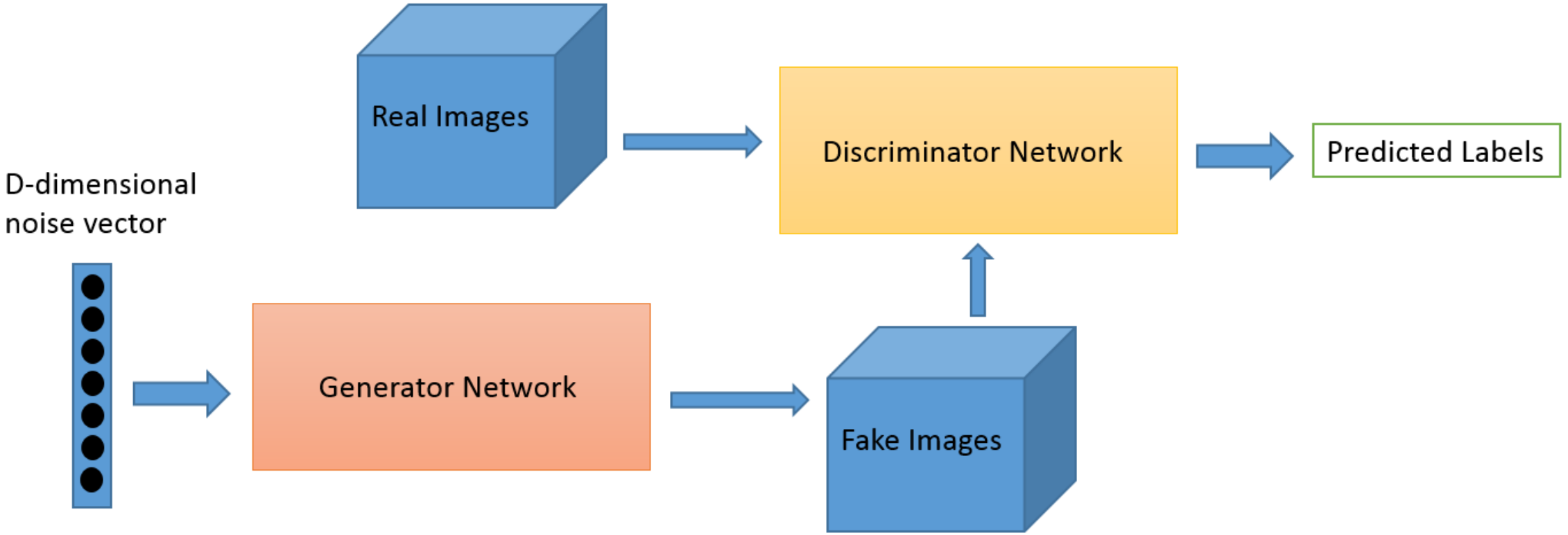
Figure 2: From left to right: bicubic interpolation, deep residual network optimized for MSE, deep residual generative adversarial network optimized for a loss more sensitive to human perception, original HR image. Corresponding PSNR and SSIM are shown in brackets. [4× upscaling]

- A Rede Generativa Adversarial (**Generative Adversarial Network**) tem recebido bastante atenção de pesquisas em IA, pelo fato de permitir a geração de dados completamente sintéticos através da rede.

- De modo geral, a GAN é uma Rede Neural que aprende a gerar imagens sintéticas, similares a um conjunto (modelo) previamente treinado.

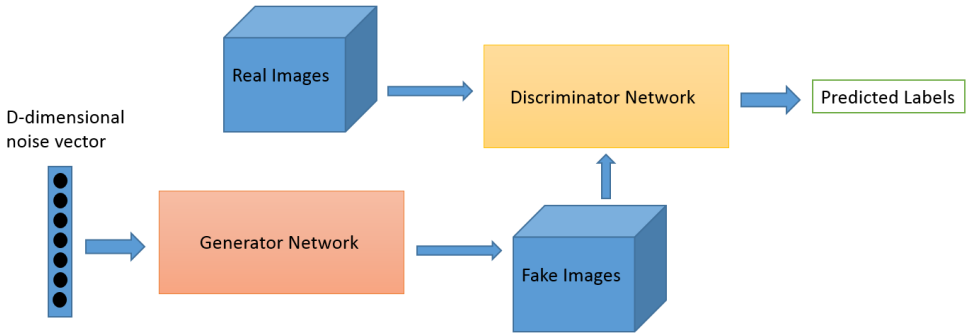
Fonte: https://www.analyticsvidhya.com/blog/2017/06/introductory-generative-adversarial-networks-gans/

Deep Learning: GAN's



Fonte: https://skymind.ai/images/wiki/gan_schema.png

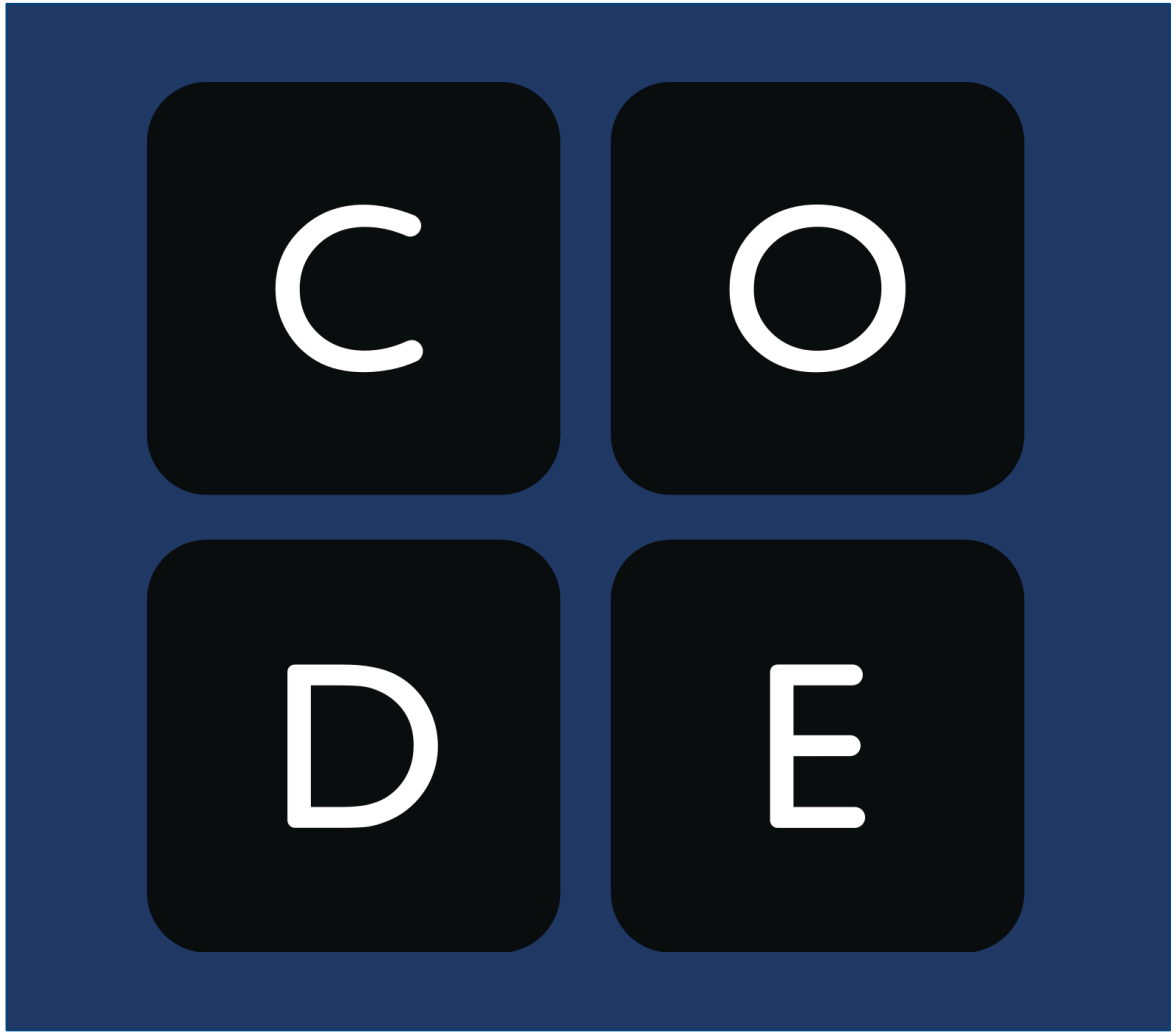
Deep Learning: GAN's



- Funcionamento:
- O Gerador recebe um conjunto de números aleatórios e gera uma imagem (ou conjunto de imagens)
- A imagem gerada (falsa) é inserida no discriminador juntamente com imagens verdadeiras, para que o discriminador realize o treinamento
- Após o treinamento, o discriminador fornece os valores de probabilidade das imagens falas para o gerador, a fim de que este faça seu próprio ajuste interno.

Fonte: https://www.analyticsvidhya.com/blog/2017/06/introductory-generative-adversarial-networks-gans/

Challenge Time





Referências



- RUSSELL, STUART J.. Inteligência Artificial. 2ª ed. São Paulo: Campus - Elsevier, 2004.
- FERNANDES, Anita M. R.. Inteligência Artificial: noções gerais. 1ª ed. Florianópolis: Visual Books, 2008. Capítulo 1.
- SILVA, I.N. – Sistemas Inteligentes – Notas de Aula da disciplina de Redes Neurais Artificiais – USP São Carlos, 2012.
- SILVA, I.N., SPATTI, D.H., FLAUZINO, R.A., Redes Neurais Artificiais para Engenharia e Ciências Aplicadas. 1ª ed. São Paulo: Editora Artiliber, 2010.
- https://indoml.com/2018/03/07/student-notes-convolutional-neural-networks-cnn-introduction/

Referências



- THIRD GENERATION NEURAL NETWORKS: DEEP NETWORKS. https://www.mql5.com/en/articles/1103
- Get busy with Word Embeddings – An Introduction. https://www.shanelynn.ie/get-busy-with-word-embeddings-introduction/
- Sequence Models. Coursera: https://www.coursera.org/learn/nlp-sequence-models/home/welcome
- The Unreasonable Effectiveness of Recurrent Neural Networks. http://karpathy.github.io/2015/05/21/rnn-effectiveness/
- Understanding LSTM and its diagrams https://medium.com/mlreview/understanding-lstm-and-its-diagrams-37e2f46f1714

Referências



- Word2Vec Tutorial - The Skip-Gram Model. http://mccormickml.com/2016/04/19/word2vec-tutorial-the-skip-gram-model/



Facens

AQUI TEM ENGENHARIA